[VNUHCM] University of Science
Faculty of [Information Technology]

Bachelor's Thesis

Accelerating Test-Time Discovery in Complex Code Generation via Execution-Guided Self-Distillation

Draft for format/framing review — contains placeholder data (§4.2.5); see `PLACEHOLDERS.md`.

Student: Mai Dang (MSSV: _____)

Supervisor: _____

Ho Chi Minh City, 2026

Abstract

Test-time training lets a language model improve on a single hard problem at inference time, but on-policy self-distillation stalls when the model rarely produces a correct attempt to learn from. This thesis studies how to accelerate test-time discovery in complex code generation via *execution-guided self-distillation*, where execution feedback (failing tests, runtime errors) is the privileged context that drives the update. We propose a *teacher-first* organization: rather than distilling the student's own failed rollout, the self-teacher generates trajectories under feedback, a verifier and a judge filter them for correctness and independence, and the student is distilled toward those filtered trajectories, placing the method between on-policy SDPO and off-policy SFT. On matched comparisons over hard LiveCodeBench problems with a 4B model, teacher-first is at least as good as the student-first baseline on every seed and never worse, and on the hardest problems it escapes the flat-reward trap that keeps student-first stuck at zero, an escape-zero pattern replicated three times. The reprompt-template formulation of the feedback affects discovery, most clearly on hard problems. A math pilot on answer-only AIME problems does not escape, and its failure is informative: the teacher copies the answer (the judge flags most trajectories as copies, roughly four in five) and transfers reasoning style rather than method. Together these results characterize a boundary, escape requires the reference to encode an in-reach method rather than only a value. Findings are preliminary and directional, bounded by small samples; a compute-matched comparison is included as placeholder data pending the real runs.

Keywords: test-time training, self-distillation, code generation, execution feedback, discovery@k, information leak.

Contents

Chapter 1

Chapter 1 — Introduction

This thesis studies how to **accelerate test-time discovery in complex code generation via execution-guided self-distillation**. The core question is practical: when a language model is given a single hard programming problem and a budget to improve itself at inference time, what is the most effective way to turn execution feedback into a better solution, and when does that strategy work?

1.1 1.1 Background

Reinforcement learning with verifiable rewards (RLVR) is the standard recipe for post-training language models on code and math, with GRPO [8] as a common instantiation. Its weakness is sparse credit assignment: a scalar outcome reward says only whether an attempt passed, not where it went wrong, and when every sampled rollout fails, the advantage collapses and learning stalls. This failure is most acute on exactly the problems that matter, the hard ones, where successes are rare.

Self-Distillation Policy Optimization (SDPO) [1] addresses this by extracting a denser signal from the *rich feedback* that verifiable environments already produce but usually discard, such as runtime errors and failing tests. The same model, conditioned on this feedback, acts as a self-teacher whose retrospective per-token distribution is distilled back into the student. At test time, SDPO can iterate on a single hard problem and improve before it ever produces a first success, measured by discovery@k, the probability of solving within the first k attempts [1, §5].

Two open issues motivate this work. First, the parent paper organizes its update *student-first*: it distills the student's own failed rollout. On a problem the bare student rarely solves, those rollouts are almost never correct, so there is little that is right to learn from. Second, Kim et al. [2] show that distilling from rich context can degrade a model by suppressing its epistemic verbalization, which raises the question of whether the feedback helps the model reason or merely teaches it to copy.

1.2 1.2 Problem

The setting is test-time training (TTT): one hard problem, a verifiable reward, and a short budget of self-improvement steps, after which the model is reset. Two design choices

govern how well the model discovers a solution. One is *how the execution feedback is organized into a learning signal*: should the model learn from its own failed attempt, or from a correct attempt produced under feedback and filtered for independence? The other is *how the execution feedback is formulated* into the reprompt the self-teacher sees, which the parent paper leaves as an explicit open question [1, §7]. This thesis takes execution feedback as the privileged context that guides self-distillation, hence *execution-guided*, and studies both choices with the goal of accelerating discovery.

1.3 1.3 Research questions

Two primary questions drive the study:

- **RQ-method.** At test time, does a teacher-first organization, which distills filtered teacher-generated trajectories, beat the student-first baseline, which distills the student's own failed rollout, at discovering solutions to hard code problems?
- **RQ1.** Does the reprompt-template formulation of the execution feedback affect discovery, and in which regime?

Two secondary, exploratory questions situate the work within the wider self-distillation literature:

- **RQ2 (exploratory).** Does self-distillation from rich context suppress epistemic verbalization at test time? Only a limited math pilot bears on this, since the code runs produce no reasoning trace.
- **RQ3 (exploratory).** How does discovery trade off against compute? This thesis quantifies the compute overhead of teacher-first and identifies the compute-matched comparison a full answer requires; the compute-to-correct Pareto itself is left to future work (§4.2.5, §6.2.4).

1.4 1.4 Contributions

The contributions are stated at the strength the evidence supports; the sample sizes are small and the claims are directional, never proofs.

1. **A teacher-first variant of execution-guided self-distillation.** A method that distills the student toward verifier- and judge-filtered trajectories generated by the self-teacher, positioned between on-policy SDPO and off-policy SFT (Chapter 3).
2. **Evidence that teacher-first accelerates discovery where on-policy SDPO stalls.** Across matched comparisons, teacher-first is at least as good as student-first on every seed and never worse, and on the hardest problems it escapes the flat-reward trap that keeps student-first stuck at zero. This escape-zero behavior is replicated three times, with one case improving the pass-rate roughly ninefold (Chapter 4).
3. **A reprompt-template effect.** The formulation of the execution feedback affects discovery, most clearly on hard problems, where a reasoning-inducing template orders above the plain anchor, which orders above a minimal one (§4.3).
4. **A value-versus-procedure boundary characterization.** A math pilot shows where the method does *not* help: when the reference encodes only an answer rather than an in-reach method, the teacher copies the answer (a copy rate near 75% is

measured), the leak transfers reasoning style but not capability, and the student stays at zero. This characterizes escape as requiring reachable capability and a method-bearing reference, not merely a correct answer (§4.5, Chapter 5).

1.5 1.5 Scope

The study is confined to the test-time training regime: single-problem self-improvement with a per-problem reset, using LoRA adapters, which is the appropriate lightweight choice for adapting to one problem at inference time. Code generation on LiveCodeBench v6 [6] is the core, carrying the main claims; an AIME 2026 math pilot serves as a contrast that characterizes the boundary of the mechanism. Results are on one 4B-scale model per domain, an anchor template for the main comparison, and a 15-step budget. No claim of universality is made.

1.6 1.6 Thesis structure

Chapter 2 reviews the background and related work: RLVR and GRPO, SDPO and its loss, test-time training and discovery@k, the reprompt-template design space, and the epistemic-suppression literature. Chapter 3 specifies the method, including the student-first baseline, the proposed teacher-first organization, the template taxonomy, the two domains, and the metrics. Chapter 4 reports the experiments: the main teacher-first versus student-first comparison and the escape-zero result, the template study, the robustness ablations, and the math pilot. Chapter 5 discusses the mechanism and the value-versus-procedure boundary. Chapter 6 states the limitations and the future experiments that would address them. Chapter 7 concludes.

1.7 In-chapter citations

Numbered per the master bibliography (`00_references.md`): [1] Hübotter et al. 2026 · [2] Kim et al. 2026 · [6] LiveCodeBench · [8] GRPO / DeepSeekMath.

Chapter 2

Chapter 2 — Background and Related Work

This chapter sets out the background the thesis builds on. It moves from the reinforcement-learning setting that motivates self-distillation (§2.1), through the SDPO method and its loss (§2.2), to the test-time regime and its metric (§2.3), the reprompt-template design space (§2.4), the epistemic-suppression critique (§2.5), and a sister self-distillation method (§2.6), closing with the gap this thesis addresses (§2.7).

2.1 2.1 RLVR and GRPO

Reinforcement learning with verifiable rewards (RLVR) is the dominant post-training recipe for code and math, where an automatic checker supplies the reward. Group Relative Policy Optimization (GRPO) [8] is a common instantiation: it samples a group of rollouts per problem and assigns each an advantage relative to the group mean, $A_{i,t} = r_i - \text{mean}_j\{r_j\}$, which is constant across the tokens of a sequence.

GRPO is attractive because it removes the separate value network of PPO-style methods: the group mean serves as the baseline, and the advantage is estimated from the spread of rewards within the sampled group [8]. This keeps the method simple and is part of why it became the default for verifiable domains.

The same simplicity is its structural weakness in credit assignment. A scalar outcome reward records only whether a rollout passed, not which tokens were responsible, so the learning signal is sparse [1]. The weakness becomes a failure when every rollout in a group fails: the group-relative advantage collapses to zero and no gradient flows, so the method cannot learn on a problem until it has already solved it at least once. Hard problems, where successes are rare, are precisely where this stalls.

A natural fix is to distill from a stronger teacher, but that requires a teacher more capable than the student, which is unavailable when the goal is to raise the capability ceiling rather than to compress a known one [1]. This tension motivates a teacher that is not external but derived from the student itself.

2.2 2.2 SDPO

2.2.1 2.2.1 The self-teacher and rich feedback

Hübotter et al. [1] observe that many verifiable environments already expose *tokenized* feedback, such as runtime errors, failing-test diffs, or judge comments, which is discarded when the signal is collapsed to a scalar reward. They propose reinforcement learning with rich feedback (RLRF) as a paradigm that keeps this signal, and SDPO as its instantiation. The mechanism that makes it possible is in-context learning: the same model, conditioned on the feedback f in addition to the question x, can often locate its own mistake. This conditioned model, $\pi_\theta(\cdot \mid x, f)$, serves as a *self-teacher*, while the unconditioned $\pi_\theta(\cdot \mid x)$ is the student. Both share the weights θ, so this is self-distillation in the lineage of knowledge distillation [13], but with feedback rather than a larger network as the source of the teacher's advantage.

2.2.2 2.2.2 The loss and dense credit assignment

SDPO distills the self-teacher's retrospective next-token distribution into the student by a per-token KL:

$$\mathcal{L}_{\text{SDPO}}(\theta) = \sum_{t=1}^{|y|} \mathrm{KL}\Big(\pi_\theta(\cdot \mid x, y_{<t}) \,\Big\|\, \text{stopgrad}\, \pi_\theta(\cdot \mid x, f, y_{<t})\Big),$$

where `stopgrad` prevents the teacher from collapsing onto the student to ignore f [1]. With the teacher detached, the gradient at a student logit is $\partial\mathcal{L}/\partial z_v = \pi_S(v) - \pi_T(v)$, so the optimizer raises every logit the teacher trusts more than the student does. The contrast with GRPO is the density of the target:

Method	Target per token	Signal per sequence
GRPO	scalar advantage × one-hot	$O(1)$
SDPO (sequence-level)	scalar × soft distribution	$O(1)$
SDPO (token-level)	soft distribution, top token	$O(T)$
SDPO (logit-level)	soft distribution over top-K vocab	$O(T \cdot K)$

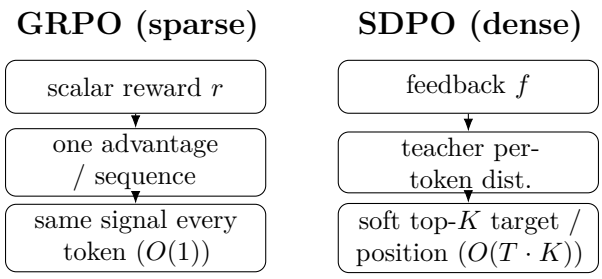


Figure 2.1 *Credit-assignment density, GRPO versus SDPO.* GRPO attaches one scalar advantage to a whole sequence, so every token receives the same signal ($O(1)$). SDPO conditions the teacher on feedback and supplies a soft, K-dimensional target at each position ($O(T \cdot K)$). Schematic; see the table above for the formal density ordering.

The full method is logit-level: the target is a K-dimensional distribution at every position rather than a single scalar per sequence, which is the source of its sample efficiency [1]. To keep this affordable at a vocabulary of roughly 150k, only the teacher's top-K logits (with K=100, or 20 for the code configuration [4]) plus a tail bucket are kept. The KL direction is parameterized by α: forward KL (α=0) is mode-covering, reverse KL (α=1) is mode-seeking, and JSD (α=0.5) interpolates, the last following Agarwal et al. [11] for stability. The self-teacher is regularized, typically by an EMA of the student weights or a trust-region interpolation, since an unregularized teacher diverges [1]. The implementation used in this thesis is specified in §3.3.5; the derivation is consolidated in the project notes.

2.2.3 2.2.3 Three regimes

The parent paper validates SDPO across three settings. In a plain RLVR setting without native rich feedback, it manufactures feedback by using a successful in-batch rollout as the reference for a failed one, and still beats GRPO substantially, reaching a GRPO five-hour accuracy in fifty minutes on one chemistry task [1]. In the RLRF setting with genuine code-execution feedback on LiveCodeBench v6 [6], it reports 48.8% versus GRPO's 41.2% and matches GRPO's final accuracy with roughly 4× fewer generations. The third setting, test-time discovery on hard problems, is the regime this thesis works in and is treated next.

2.2.4 2.2.4 What drives SDPO: ablations and scaling

Three findings from the parent paper's analysis bear directly on the choices made in this thesis.

First, both ingredients of SDPO matter. Decomposing the method into logit-level, token-level, and sequence-level variants gives an ordering logit > token > sequence > GRPO [1, §4.2], which shows that rich feedback and dense credit assignment each contribute rather than one carrying the result. A feedback-type ablation (its Table 6) is equally pointed: the environment output alone reaches 39.9% training accuracy and the model's own prior solution alone 42.6%, while combining them reaches 48.3%, so the two are complementary. Adding the student's own *failed* attempt to the teacher context, by contrast, drops accuracy to 44.5% and lowers entropy by 0.23, because the teacher is biased toward the student and loses diversity. This last result is part of the motivation for the teacher-first organization of §3.3, which distills filtered teacher generations rather than the student's failed attempt.

Second, self-teaching is emergent with scale [1, §4.1]. On Qwen3-8B, SDPO far exceeds GRPO; on Qwen3-0.6B the two are comparable; and on Qwen2.5-1.5B, SDPO *underperforms* GRPO. The method needs in-context learning strong enough for the conditioned model to act as a useful teacher. This places the 4B model used here in a band where SDPO is expected to work, and it also predicts that on a stronger model the student-first baseline should improve on its own, narrowing the teacher-first margin, a prediction revisited in Chapter 6.

Third, the self-teacher is bootstrapped, not fixed. It updates alongside the student and its accuracy rises during training, and the final student exceeds the *initial* teacher, so the method is not capped by the teacher it starts with [1, §4.3]. Regularization is what keeps this stable: a trust-region teacher (50.6%) edges out an EMA teacher (49.3%) and a

frozen one (48.8%), while an unregularized teacher diverges. SDPO also forgets less than supervised fine-tuning on the self-teacher's outputs across held-out benchmarks, though slightly more than GRPO, indicating that the on-policy character of the update helps preserve prior capability [1, §4.4]. A hybrid that blends GRPO and SDPO advantages helps weak models but not strong ones [1, §4.5].

2.3 2.3 Test-time training and discovery@k

In the test-time setting [1, §5], a single hard problem is fixed, the reward is binary, and the model must discover a solution as quickly as possible. Rather than concatenating a growing history into the context, as multi-turn reprompting does, SDPO compresses the feedback into the weights by distilling $\pi_\theta(\cdot \mid x, c)$ into $\pi_\theta(\cdot \mid x)$, which sidesteps the context-window limit.

The metric is discovery@k [1, Def. 5.1], the probability of solving within the first k attempts, $P(T \leq k)$ with discovery time $T = \min\{k : r(y_k) = 1\}$. It generalizes pass@k [9] from i.i.d. sampling to sequential algorithms that share state or update weights between attempts, and it reduces to pass@k when the algorithm is plain best-of-k. The lineage is the time-to-termination performance profile of Dolan and Moré [12]. Empirically, SDPO discovers 53.2% of very-hard problems against best-of-k's 41.5%, and on some problems is the only method to solve at all, even though the initial self-teacher accuracy is below 1% on almost every hard problem [1]. That last point matters most: dense credit assignment lets the model improve before it has produced a single success.

2.4 2.4 The reprompt template and its design space

The self-teacher's behavior is controlled by the template that formats x and f into its context. The parent paper uses a slot-based template (its Table 2) that inserts a successful previous rollout, the environment feedback, and a closing directive, and reports that performance is "not sensitive to syntactic variations" of this template, while the *feedback type* does matter: an ablation (its Table 6) finds the environment output and the model's own solution to be complementary, and including the student's own failed attempt in the teacher context reduces diversity [1].

Two gaps follow. The paper tests syntactic variation but not semantic or instructional variation, and it explicitly names systematic study of how the reprompt template influences behavior as future work [1, §7]. This thesis organizes the template design space along three dimensions: information content, motivated by Kim et al.'s finding that context richness governs suppression [2]; instruction framing, addressing the open question above; and memory depth, motivated by the sequential accumulation of the test-time setting. The seven templates of §3.4 sample points across these dimensions, with the paper's default as the anchor.

2.5 2.5 Epistemic verbalization and suppression

Kim et al. [2] supply the cautionary counterpart to SDPO. They show that distilling from a rich context can degrade a model by suppressing its *epistemic verbalization*, the uncertainty markers such as "wait" or "maybe" that accompany genuine reasoning. Their

analysis ties the magnitude of suppression to the mutual information $I(y; c \mid x)$ between the output and the context: the richer the context, the stronger the suppression. They probe the spectrum by varying what the teacher context contains, from nothing ($c = \varnothing$), through a solution stripped of its thinking and a regenerated answer, to a full solution, and find suppression growing with the information the context carries. The effect accumulates across successive distillation steps, and they recommend freezing the teacher (no EMA update) to break the feedback loop, in tension with the parent paper's preference for a moving, regularized teacher (§2.2.4).

For this thesis the relevance is twofold. The suppression risk is sharpest exactly when the context contains the answer, which is the math leak regime studied in the pilot (§4.5), where the teacher can copy the answer rather than derive it. And the accumulation across steps is precisely what the test-time setting applies repeatedly, since each TTT step distills again from the same answer-bearing context.

2.6 2.6 SDFT and the leak concern

Shenfeld et al. [3] propose self-distillation fine-tuning (SDFT) for continual learning, an on-policy, minimum-change distillation toward the model's own conditioned generation. It is a sister method: like SDPO it uses the model as its own teacher, but its aim is to preserve prior capability while adapting, which makes minimizing unintended change a design goal. The teacher-first organization of this thesis sits near SDFT in that it distills toward filtered model-generated trajectories, but it uses execution feedback as the conditioning context rather than a fixed demonstration (§3.3.1). SDFT's minimum-change framing also sharpens the leak question: if distillation transfers more than the intended content, what exactly is being copied, which is the question the math pilot answers concretely.

2.7 2.7 The gap this thesis addresses

The parent paper studies a student-first, on-policy SDPO, focuses its analysis on the train-time regimes, and reaches the test-time setting without studying the reprompt template there or measuring epistemic behavior. Kim et al. characterize epistemic suppression but not in the test-time code-discovery setting. Shenfeld et al. minimize change for continual learning rather than maximize discovery on a single hard problem. The gap, then, is threefold. First, a teacher-first organization of execution-guided self-distillation that distills filtered teacher generations, positioned between SDPO and SFT, and the question of whether it accelerates test-time discovery (RQ-method). Second, the effect of the reprompt-template formulation on that discovery, the open question the parent paper names (RQ1). Third, a characterization of *when* the approach helps, framed as the value-versus-procedure boundary that the code-versus-math contrast exposes. The remainder of the thesis addresses these in turn.

2.8 In-chapter citations

Numbered per the master bibliography (`00_references.md`): [1] Hübotter et al. 2026 · [2] Kim et al. 2026 · [3] Shenfeld et al. 2026 · [4] lasgroup SDPO repo · [6] LiveCodeBench

· [8] GRPO / DeepSeekMath · [9] Chen et al. 2021 · [11] Agarwal et al. 2023 · [12] Dolan & Moré 2002 · [13] Hinton et al. 2015.

Chapter 3

Chapter 3 — Method

This chapter specifies the method in enough detail to reproduce it. The object of study is **test-time SDPO** [1, §5]: a single hard problem is fixed, the model improves itself over a small number of training steps at inference time (test-time training, TTT), and we measure how well it *discovers* a solution. Because the privileged context that drives the update is **execution feedback** (failing tests, runtime errors), we call this regime **execution-guided self-distillation**, and the goal is to *accelerate* that test-time discovery in complex code generation. Within the regime, the thesis contrasts two ways of organizing the update, **student-first** (the baseline of the originating paper [1]) and **teacher-first** (a variant proposed by the advisor), and then studies how the reprompt template formulates the execution feedback (RQ1).

Domain scope is asymmetric by design. **Code generation is the core** (LiveCodeBench v6 [6]); **math is a pilot and a contrast** (AIME 2026 [10]), used to characterize the *boundary* of the mechanism rather than to prove it works everywhere. That split is kept consistent throughout: every math setting carries the corresponding model and reference caveats (§3.5, Chapter 6).

Notation mirrors the parent paper, Hübotter et al. [1], so the two can be read side by side.

3.1 3.1 Test-time SDPO pipeline

3.1.1 3.1.1 Notation and setting

Fix a problem x (the problem statement). Two policies share the same weights θ:

- the **student** $\pi_\theta(\cdot \mid x)$, which sees only the problem, and
- the **self-teacher** $\pi_\theta(\cdot \mid x, c)$, the same model conditioned on an additional **privileged context** c.

In SDPO, c plays the role of the "feedback" f in the parent paper: retrospective information (a runtime error, a failing test, or a reference solution) that lets the model look back and locate its mistake [1, §2]. Because teacher and student differ only in their input context, this is genuine *self*-distillation: the model learns by pulling its next-token

distribution (without context) toward its own distribution conditioned on c.

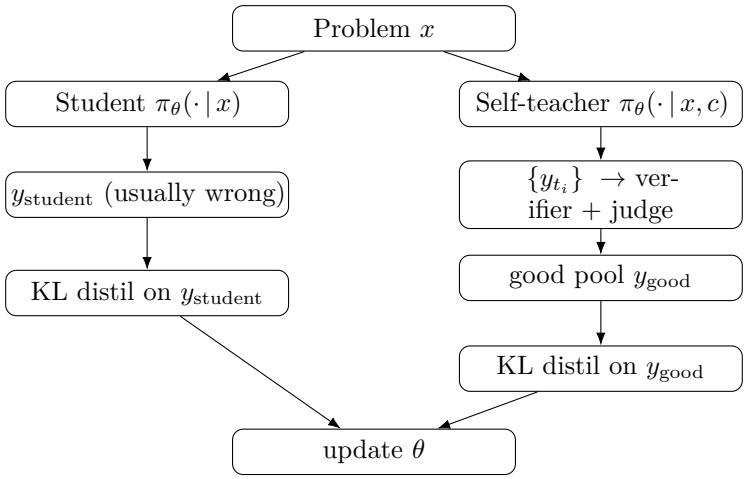
The test-time setting follows Hübotter et al. [1, §5]. For each problem x, the model is reset to a base checkpoint and then runs K TTT steps *on that problem alone*. The reward is verifiable, graded for code and binary for math. The goal is not to generalize to other problems but to **discover** a solution to x as early as possible; the metric is discovery@k [1, Def. 5.1] (§3.6).

3.1.2 3.1.2 The shared loop

Both arms share one skeleton. The single thing that differs between them is *which trajectories are distilled*:

```
load base model + LoRA + optimizer (AdamW, lr = 1e-5)
PRE-eval (student pi_theta(*|x) only, N_eval samples)        # baseline
for step in 1..K:
    feedback        <- build_feedback(student attempt, verifier)   # rich feedback
    {trajectories}  <- generate(...)                               # arm-specific
    {distill_targets} <- select(...)                               # arm-specific (the
↪  core difference)
    if distill_targets != {}:
        theta <- theta - lr * grad_theta L_KL(distill_targets)                    #
↪ one optimizer step
    log step stats -> W&B
POST-eval (student pi_theta(*|x) only, N_eval samples)       # after TTT
report PRE->POST and the discovery curve
```

Figure 3.1 sketches the two arms; they diverge only at the shaded block.



The whole contrast reduces to one sentence: **student-first distills the student's own failed attempt; teacher-first distills a correct, filtered trajectory generated by the teacher.** Everything else (loss, model, hyperparameters) is held fixed to isolate that one variable.

3.2 3.2 Student-first SDPO (baseline, on-policy)

This is the original algorithm of Hübotter et al. [1], realized through the `SDPOTrainer` of TRL [5] (script `07_discovery_curve.py`).

At each step the student samples a rollout $y \sim \pi_\theta(\cdot \mid x)$ on-policy. The verifier grades y and produces feedback f. The self-teacher $\pi_\theta(\cdot \mid x, f)$ then rescores *every token* of that same y: given f, what should the correct next-token distribution have been at each position? The student is pulled toward this retrospective distribution by a per-token KL:

$$\mathcal{L}_{\text{SDPO}}(\theta) \;=\; \sum_{t=1}^{|y|} \mathrm{KL}\Big(\pi_\theta(\cdot \mid x, y_{<t}) \;\Big\|\; \text{stopgrad}\, \pi_\theta(\cdot \mid x, f, y_{<t})\Big).$$

The `stopgrad` blocks gradient flow through the teacher, which prevents the teacher from collapsing onto the student and ignoring f [1, §2]. The gradient and the top-K approximation are shared with teacher-first and are deferred to §3.3.5.

There is a failure mode on hard problems that motivates the whole next section. Because rollouts are on-policy, on a problem the *bare* student rarely solves, almost every rollout is wrong, so the feedback is poor and the distillation signal is weak and unstable. This is the **flat-reward trap**: there is no correct rollout to learn from. Hübotter et al. [1, §5] show SDPO can still iterate from rich feedback even when initial teacher accuracy is below 1%, but they use Qwen3-8B; on the weaker 4B model used here (§3.5), the stuck-at-zero behavior does appear, and it is exactly what teacher-first targets.

3.3 3.3 Teacher-first SDPO (proposed)

3.3.1 3.3.1 Motivation

Two problems with student-first motivate the variant.

First is the flat-reward trap of §3.2: when the student never rolls a correct attempt, there is nothing correct to distill.

Second is **information leak** [2]. If c contains a reference solution, the teacher rescores toward "close to the reference," so across many test-time steps the student converges onto *copying* the reference rather than learning to reason. Kim et al. [2] formalize this: when $I(y; c \mid x)$, the information richness of the context, is high, the model **suppresses epistemic verbalization** and degrades.

The core change is to **reverse the order**. The teacher generates first; correct, independent trajectories are filtered out; and the student is distilled toward *those*. The KL is computed on y_{good} (teacher-generated, filtered), not on y_{student} (a failed attempt):

$$\mathcal{L}_{\text{TF}}(\theta) \;=\; \sum_{y \in \mathcal{G}} \sum_{t=1}^{|y|} \mathrm{KL}\Big(\pi_\theta(\cdot \mid x, y_{<t}) \;\Big\|\; \text{stopgrad}\, \pi_\theta(\cdot \mid x, c, y_{<t})\Big),$$

where $\mathcal{G}$ is the good pool (§3.3.3). Conceptually, teacher-first sits **between SDPO (on-policy) and SFT (off-policy)**. It resembles SDFT [3] in that it distills toward generations produced by the (context-conditioned) model itself, but the context is SDPO-style *feedback* [1] rather than a fixed demonstration.

Because `SDPOTrainer` hard-codes the student-first rollout internally, teacher-first is implemented as a **custom training loop** (script `09_teacher_first.py`) that reuses the helpers of `07` (`build_privileged_context`, `build_dynamic_feedback`, `evaluate_solution`, evaluation).

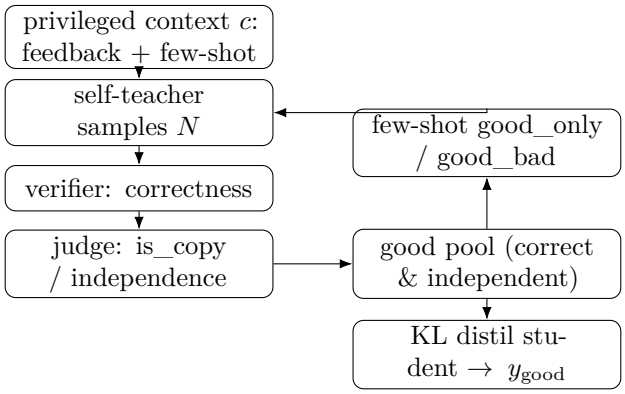


Figure 3.2 *The teacher-first step.* The self-teacher generates under the privileged context, a verifier and a judge filter trajectories into a good pool, the good (and optionally bad) exemplars steer the next teacher rollout in-context, and the student is distilled only toward the filtered good trajectories. The few-shot loop trains no teacher weights.

3.3.2 3.3.2 Teacher rollout and privileged context

At each step the teacher samples N trajectories (`teacher_n`) at high temperature for diversity:

$$\{y_{t_1}, \ldots, y_{t_N}\} \sim \pi_\theta(\cdot \mid x, c), \qquad c = \underbrace{f}_{\text{feedback}} + \underbrace{\text{few-shot exemplars}}_{\S 3.3.4}.$$

The `privileged_context` c is the concrete realization of "feedback" from [1]; the name belongs to the thesis codebase, the concept to the paper. The content of c is domain-dependent (§3.5). The few-shot block is inserted *before* the final directive ("Respond with ONLY…"), with at most `max_fewshot = 3` exemplars so the context does not overflow, and the token count is logged each step.

3.3.3 3.3.3 Verifier and judge produce the good pool

Each trajectory y_{t_i} is labeled in two stages.

Stage 1, the verifier (correctness). For code (LCBv6), `evaluate_solution` runs the public and private tests and returns the fraction of tests passed, a **graded** reward in $[0, 1]$ that gives a usable gradient to climb (for example 0.21 then 1.0). For math (AIME), an answer match gives a **binary** reward in $\{0, 1\}$.

Stage 2, the judge (independence). This measures whether y_{t_i} merely *copies* the reference:

$$\text{good} := (\text{score} \geq 1.0) \wedge (\text{independent}), \qquad \text{bad} := (\text{copies the reference}).$$

Two judge implementations are used and later ablated (§4.4):

Judge	Mechanism	Cost
`difflib`	`SequenceMatcher` string similarity on normalized code (comments and whitespace stripped); copy if similarity $\geq$ threshold ($\approx$0.9)	cheap, deterministic
LLM	groq `llama-3.3-70b` (code) or `glm-4.5-flash` (math), a derive-vs-copy prompt returning `is_copy` and `reasoning_quality` 1–5	API cost, semantic

> **An honest note on the judge.** With thinking OFF (code) there is no reasoning trace to grade, and `reasoning_quality` saturates at 4–5. The LLM judge therefore acts purely as a **semantic copy-detector**, not as a reasoning-quality scorer. The thesis does not claim it measures reasoning quality.

There is one further mechanism worth stating up front because it becomes a measured limitation. When no trajectory clears Stage 2 (every candidate is flagged as a copy), `filter_trajectories` falls back to keeping the single best *correct* trajectory as good, so the loop does not stall on a cold start. The fallback is necessary to avoid an empty good pool, but it can **defeat the judge** when the teacher only ever produces copies: a copy the judge rejected is still forced into the pool. This is confirmed in the math log for idx8 (§4.5.4, Chapter 6); it is flagged here so the mechanism is transparent.

3.3.4 3.3.4 Few-shot teacher steering (good_only vs good_bad)

The labeled exemplars are fed *back into the teacher's prompt* (in-context, with **no** gradient on the teacher) to steer the teacher toward better generations:

	Option 2 — `good_only`	Option 1 — `good_bad`
Mechanism	positive few-shot (good exemplars only)	contrastive few-shot (good + bad, labeled)
Steering strength	moderate	stronger
Leak	**clean** (good is filtered, hence independent of the reference)	**residual** (bad $\approx$ reference, so reference-like content re-enters the context)

Since `bad` $\approx$ `reference`, putting bad exemplars into the prompt (even labeled "bad") shows the teacher something close to the reference again. The ablation of Option 1 versus Option 2 is therefore simultaneously an ablation of **leak versus no-leak**, a direct test of the advisor's concern. The result is a leak-null on this data (§4.4).

3.3.5 3.3.5 The distillation step: token-aligned top-K reverse KL

This is the computational core, shared by both arms; only the target trajectory differs (y_{student} versus y_{good}).

Per-token distributions. With logits $z \in \mathbb{R}^V$ ($V \approx 150$k), $\pi(v) = \text{softmax}(z)_v$. At each position t of the target trajectory we have $\pi_S = \pi_\theta(\cdot \mid x, y_{<t})$ (student) and $\pi_T = \pi_\theta(\cdot \mid x, c, y_{<t})$ (teacher, conditioned on c).

The core gradient. With the teacher detached, the loss gradient with respect to a student logit is [1; derivation in `con_sdpo_loss_mechanics`]:

$$\frac{\partial \mathcal{L}}{\partial z_v^S} \;=\; \pi_S(v) - \pi_T(v).$$

Wherever the teacher trusts a token more than the student ($\pi_T > \pi_S$), the optimizer raises that logit. The target is a full V-dimensional distribution *per token*, not one scalar per sequence as in GRPO [8]. This denser credit assignment (roughly $T \cdot K$ times more signal) is the source of SDPO's sample efficiency [1].

KL direction (α). The codebase [4] parameterizes the divergence through $\alpha \in [0, 1]$ with mixture $M = (1-\alpha)\pi_S + \alpha\pi_T$: $\alpha = 0$ is forward KL (mode-covering, preserves epistemic tokens), $\alpha = 1$ is reverse KL (mode-seeking, prone to suppression), and $\alpha = 0.5$ is JSD. The thesis uses $\alpha = 1.0$ (reverse KL, top-K = 20) to match the LCBv6 configuration of [4]. Whether α modulates suppression is an open RQ2 question (Chapter 6).

Top-K approximation. Full-vocabulary KL is too memory-heavy at $V \approx 150$k, so we use the teacher's top-20 logits plus one tail bucket; the student is projected onto the same K indices, and KL is taken over the resulting $(K+1)$-dimensional distribution [1, §2.2].

Importance sampling. Samples are drawn under θ_{old} but the loss is taken under the current θ, so each per-token loss is scaled by a clipped ratio $r_t = \min(\pi_\theta / \pi_{\theta_{\text{old}}}, c_{\text{clip}})$ with $c_{\text{clip}} = 2.0$.

Token alignment, the easiest place to get it wrong. The student prompt (`x + ↪ y_good`) and the teacher prompt (`x + c + y_good`) have *different* prefix lengths, so the logits at the positions of y_{good} must be extracted on *each* side before the KL is taken. An off-by-one here corrupts the KL silently, so the code asserts that the two lengths match and logs `(student_prefix_len, teacher_prefix_len)`. The sanity check passed: alignment is correct (L matches on both sides despite different prefixes) and the token-mean KL is finite and reasonable.

Hyperparameters: AdamW `lr = 1e-5`, `kl_topk = 20`, `kl_alpha = 1.0`, `is_clip = 2.0`. The teacher is always detached (self-distillation with shared weights, so the teacher is the student forward pass under context c, run under `no_grad`).

3.4 3.4 Reprompt templates (T1–T7)

The reprompt template determines the content of c that the teacher sees, which directly changes π_T and so the direction of the distillation gradient. It is the central variable of RQ1. To make the choice defensible (the obvious reviewer question is "why these seven?"), we do not justify each template in isolation. Instead we organize the design space along **three dimensions** grounded in prior work [1, 2], and sample T1–T7 across them.

Template	Name	Dimension	Varies from T2 by
T1	Minimal	Information content (low)	drop structure, just "incorrect, try again"
T2	**Standard (anchor)**	**baseline**	**failing test + expected + actual + error_type**
T3	Verbose	Information content (high)	T2 + full stack trace + previous code
T4	Structured JSON	Instruction framing (format)	same content as T2, JSON-encoded
T5	Reasoning-inducing	Instruction framing (diagnostic)	T2 + "First, identify root cause, then fix."
T6	First-person	Instruction framing (reframe)	"I attempted this and got X. Let me reconsider."
T7	Cumulative history	Memory depth	all N prior attempts, not just the latest

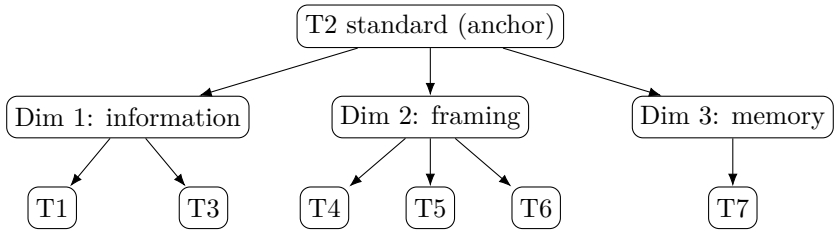


Figure 3.3 *The reprompt-template design space.* Each template varies one dimension away from the T2 anchor: information content (T1, T3), instruction framing (T4, T5, T6), or memory depth (T7). The experiments probe T1/T2/T5 (§4.3).

The three dimensions and their grounding:

1. **Information content** (T1 → T2 → T3). Kim et al. [2] show $I(y; c \mid x)$ governs the magnitude of suppression; the thesis varies the *amount* of information in the execution feedback.
2. **Instruction framing** (T4, T5, T6). Hübotter et al. [1, §4.6] conclude that small *syntactic* variation does not matter, but *semantic/instructional* variation was never tested; [1, §7] names this as future work almost verbatim ("how ... the reprompt template ... influences behavior").
3. **Memory depth** (T7). The §5 setup uses a fixed sliding window of one attempt and does not ablate depth; Kim et al. [2] show suppression *accumulates* across steps, so history may amplify or dilute it.

T2 is the anchor: every other template varies exactly one dimension and holds the rest fixed, giving a clean ablation. Of the seven, **four are implemented** (T1, T2, T5, T6; verbatim strings in Appendix B) and three (T3, T4, T7) remain conceptual points in the taxonomy. Under the available compute, RQ1 probes **T1/T2/T5** (§4.3); T6 is implemented but unprobed, and T3/T4/T7 are left to future work (Chapter 6). Two caveats are acknowledged: T4 (JSON) also changes information density (an overlap of dimensions 1 and 2), and T7 increases context length and so confounds with compute cost, which must be reported separately.

3.5 3.5 Domains: code (core) and math (pilot)

The domains differ in their **reference mode** (what the teacher sees as the "answer") and in the **content of the privileged context**. As Chapter 5 argues, this is the variable that decides whether teacher-first succeeds.

	Code — LCBv6 [6] (core)	**Math — AIME 2026 [10]** (pilot)
Model	Qwen3-4B [7], LoRA r=32, thinking **OFF**	Gemma-4-E4B, LoRA all-linear, thinking **ON**
Verifier	public/private tests → **graded** $[0, 1]$	answer match → **binary** $\{0, 1\}$
`reference_mode`	`best_in_batch` (best-scoring trajectory as proxy)	`ground_truth` (teacher always sees the correct answer)
Leak regime	**null** (reference is best-in-batch + public tests, not a reference solution)	**extreme** (feedback is "the correct answer is {gt}")
`privileged_context`	execution feedback (failing test, expected/actual, error type) + the correct best-in-batch trajectory	the correct answer (a number); the teacher need only copy `\boxed{}`
Reference contains a method?	**Yes** (best-in-batch is a *program* = an algorithm, generalizing to new inputs)	**No** (only a *value*, not a derivation)

A note on the `reference_text` choice for code: LCBv6 has no reliable reference-solution field, so the thesis uses **best_in_batch** (the highest-scoring trajectory in the teacher's batch each step) as the reference against which the judge measures independence. This has an important side effect. Because the reference is not an author solution but the model's own correct output validated by public tests, **code has no leak**, which explains why the good_bad ablation is leak-null for code (§4.4) while leak is measurable for math (§4.5).

The TTT budget also differs between domains (full values in Appendix A): code runs **15 steps, eval 16 samples, teacher_n = 10**; math runs **3 steps, eval 4 samples, teacher_n = 4, max_new_tokens = 16384** (8192 truncated the `\boxed` answer and produced spurious zero scores). This budget gap is a variable to keep in mind when comparing the two domains, alongside model and reference.

Two math caveats travel with every math result:

1. **Model confound.** Code uses Qwen3-4B (which escapes) while math uses Gemma-4-E4B, a *different* reasoner. "Math no-escape" may be partly a property of the model, not the domain.
2. **Answer-only reference.** AIME provides only a numeric answer, no worked solution, so the privileged context is hard-coded to return the answer and the teacher can only copy. MATH-500 (which has a `solution` field) is the route to removing this confound (Chapter 6) and is out of scope for the present results.

Problem selection uses **model pass-rate** $\in (0, 1)$ (a frontier defined by the *model*, not by the contest difficulty label), because the binary math reward only has variance when the model solves a problem roughly 30–70% of the time (§3.6, §4).

3.6 3.6 Metrics and comparison design

pass@k / pass_rate. We report an empirical estimate of pass@k [9]: for each condition we draw `N_eval` samples (16 for code, 4 for math) and report `pass_rate`, the fraction correct. PRE (before TTT) and POST (after TTT) are reported, evaluating the **student only**, $\pi_\theta(\cdot \mid x)$, with no context at evaluation time (otherwise the context would leak into the metric). A single deterministic `greedy` sample is reported alongside as a noisy secondary signal.

Discovery curve. Following discovery@k [1, Def. 5.1] from §3.1: the probability of solving within the first k attempts, $P(T \leq k)$ with $T = \min\{k : r(y_k) = 1\}$. Because test-time generation is *sequential* (shared state, weight updates), i.i.d. pass@k does not apply, and discovery@k is the correct metric. We log the per-step batch pass-rate as a coarse discovery curve.

Escape-zero (the mechanism evidence). Operational definition: a seed where **student-first stays stuck at pass = 0** (or drops back to 0) while **teacher-first escapes 0**. This is the direct signature of the flat-reward trap (§3.2): on-policy has no correct rollout to learn from, while the off-policy-leaning teacher can inject a correct solution. Replicated escape-zero is the main mechanistic contribution (§4.2).

Matched comparison. The two arms run on the *same* base and the *same* seed, so PRE matches per seed, giving a **paired (matched)** comparison that removes base- and seed-level variance. On each matched pair we count wins, ties, and losses (TF versus SF).

> **An honest note on small n.** Because the number of problems and seeds is small (n = 2–3 problems for code, n = 2 seeds for RQ1, n = 2 problems for math), the thesis reports only a **crude sign test** (for example 0.5^k over the non-tied pairs) as *descriptive, directional* evidence, never as an inferential proof. The language stays at "preliminary / directional / weak dominance"; we do not write "we prove." Every p-value is paired with a caveat about n (Chapter 4, Chapter 6).

3.7 In-chapter citations

Numbered per the master bibliography (`00_references.md`): [1] Hübotter et al. 2026 · [2] Kim et al. 2026 · [3] Shenfeld et al. 2026 · [4] lasgroup SDPO repo · [5] TRL v1.4.0 docs · [6] LiveCodeBench · [7] Qwen3 · [8] GRPO / DeepSeekMath · [9] Chen et al. 2021 (pass@k) · [10] MathArena AIME 2026.

Chapter 4

Chapter 4 — Experiments

This chapter reports the empirical results. The structure follows the framing of Chapter 3: code is the core and carries the main claims, math is a pilot that characterizes the boundary of the mechanism.

The experiments answer three questions. First, the method question (RQ-method): does the proposed teacher-first organization beat the student-first baseline at test-time discovery, and if so where and how much (§4.2)? Second, RQ1: does the reprompt template matter, and in which regime (§4.3)? Third, two robustness checks: is the result invariant to the judge choice, and does the leak ablation behave as predicted (§4.4)? Section 4.5 then reports the math pilot, which does not escape and is presented as a boundary characterization rather than a failed attempt.

A standing caveat applies to every number below. The sample sizes are small (2–3 problems for the code escape results, 2 seeds for RQ1, 2 problems for math). All significance values are **crude sign tests** of the form 0.5^k over non-tied matched pairs, reported as descriptive and directional evidence, never as inferential proof. The language throughout stays at "weak / directional dominance"; nowhere is a claim stated as "proven."

4.1 4.1 Experimental setup

Models. Code uses Qwen3-4B [7] with LoRA (r=32, bf16), thinking OFF. Math uses Gemma-4-E4B with LoRA on all linear layers, thinking ON. The model differs across domains, which is a confound carried explicitly into every math claim (§4.5, Chapter 6).

Hardware. Code runs on Colab L4 (22 GB) and A100; math runs on Modal A100-80GB (Colab compute units were exhausted).

TTT protocol. Per problem, the model is reset to base, then run for K steps; the student is evaluated PRE and POST with no context (§3.6). Code: $K = 15$, eval 16 samples, `teacher_n=10`. Math: $K = 3$, eval 4 samples, `teacher_n=4`, `max_new_tokens=16384`. The distillation core is identical to §3.3.5 (AdamW lr=1e-5, top-K=20, $\alpha = 1.0$ reverse KL, IS clip 2.0). Full hyperparameters are in Appendix A.

Seeds and matching. Code uses 4 seeds (0–3). Both arms share base and seed, so PRE

matches per seed, giving a paired (matched) comparison (§3.6).

Problem selection. Problems are chosen by *model* pass-rate $\in (0, 1)$ (a model-defined frontier), not by the contest difficulty label, because a problem the model already solves or never solves carries no learning signal. The anchor template is T2 unless stated otherwise.

4.2 4.2 RQ-method: teacher-first vs student-first (code)

4.2.1 4.2.1 Frontier scan and the comparison design

A frontier scan over LCBv6 problems (idx0–90) by model pass-rate identifies problems in the learnable band. Two are used for the main matched comparison: idx39 (abc393_d, labeled hard, base pass ≈ 0.1) and idx12 (abc389_b, labeled easy but with model pass ≈ 0.12, so *model*-hard). Both arms run 4 seeds at eval-16 with PRE matched per seed.

4.2.2 4.2.2 Main matched result

On idx39, teacher-first (TF) is at least as good as student-first (SF) on every seed and strictly better on two:

seed	PRE	TF POST	SF POST	TF − SF
0	0.000	**0.438**	0.188	+0.250
1	0.000	0.062	0.000	+0.062
2	0.062	0.125	0.125	0
3	0.188	0.062	0.062	0
mean		**0.172**	**0.094**	**+0.078**

On idx12, TF reaches 1.0 on all four seeds while SF wavers:

seed	PRE	TF POST	SF POST	TF − SF
0	0.000	1.000	0.938	+0.062
1	0.062	**1.000**	0.500	+0.500
2	0.062	1.000	1.000	0
3	0.125	1.000	0.938	+0.062
mean		**1.000**	**0.844**	+0.156

Aggregating the two problems gives 8 matched seed-comparisons:

	TF ≥ SF	TF > SF	tie	TF < SF
idx39 (hard)	4/4	2	2	0
idx12 (model-hard)	4/4	3	1	0
total	**8/8**	**5**	**3**	**0**

Teacher-first is $\geq$ **student-first on all 8 seeds, strictly better on 5, never worse**, and lower-variance (TF hits 1.0 on every seed of idx12 while SF ranges 0.5–1.0). The

crude sign test over the 5 non-tied pairs is $0.5^5 \approx 0.03$. The pattern repeats across two independent problems, which argues against pure sampling noise. The honest reading is **weak dominance**: many ties, modest typical margins, and the large wins are seed-specific.

A control observation supports that the effect is real rather than an artifact of TF being more aggressive. On idx39 seed 3, where PRE is already high (0.188), *both* arms degrade identically. This is over-distillation when the base starts strong, and it applies to both arms equally, so it is not a TF-specific failure.

The aggregate means are shown in Figure 4.1 and the per-seed trajectories in Figure 4.2.

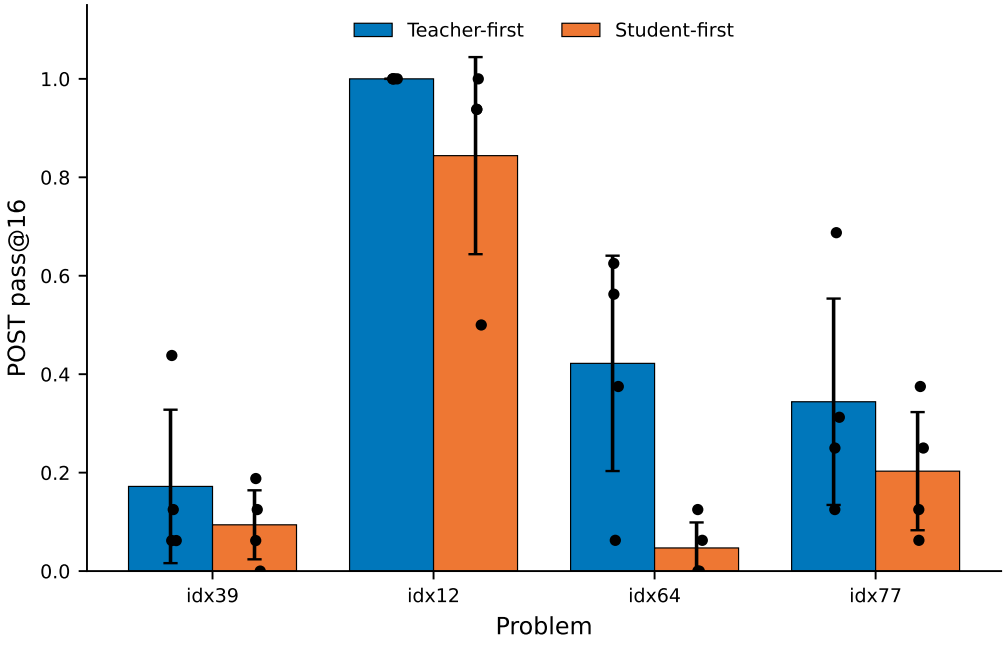


Figure 4.1 *POST pass@16, teacher-first versus student-first.* Bars are the published means; dots are per-seed values ($n = 4$, all four problems, recovered and verified from the W&B export); error bars are ± 1 SD. Teacher-first is at or above student-first on every problem.

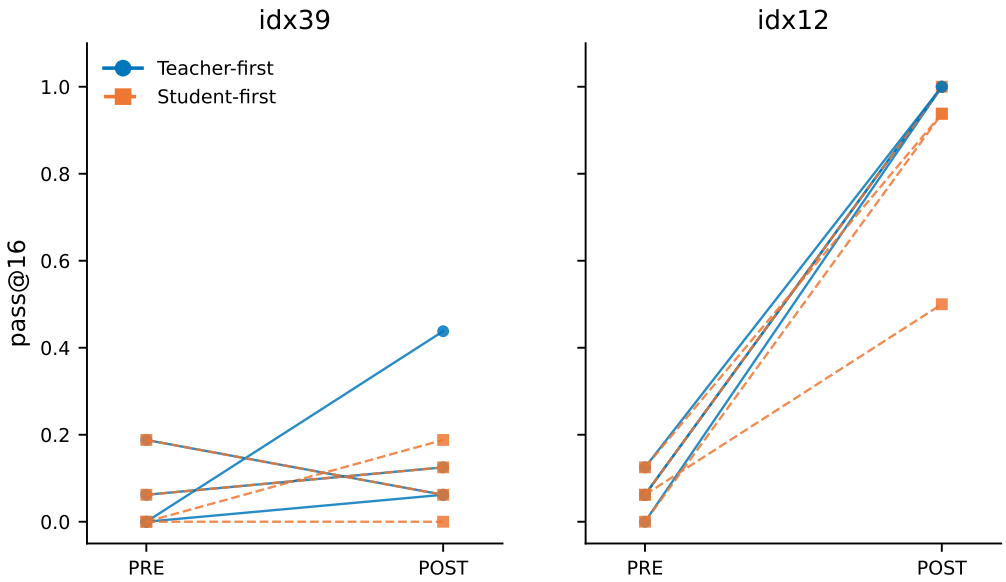


Figure 4.2 *Per-seed PRE→POST change.* Each line is one seed. Teacher-first (solid) is at or above student-first (dashed) on every seed; teacher-first reaches 1.0 on all seeds of idx12 while student-first varies.

4.2.3 4.2.3 Hard-frontier escape-zero

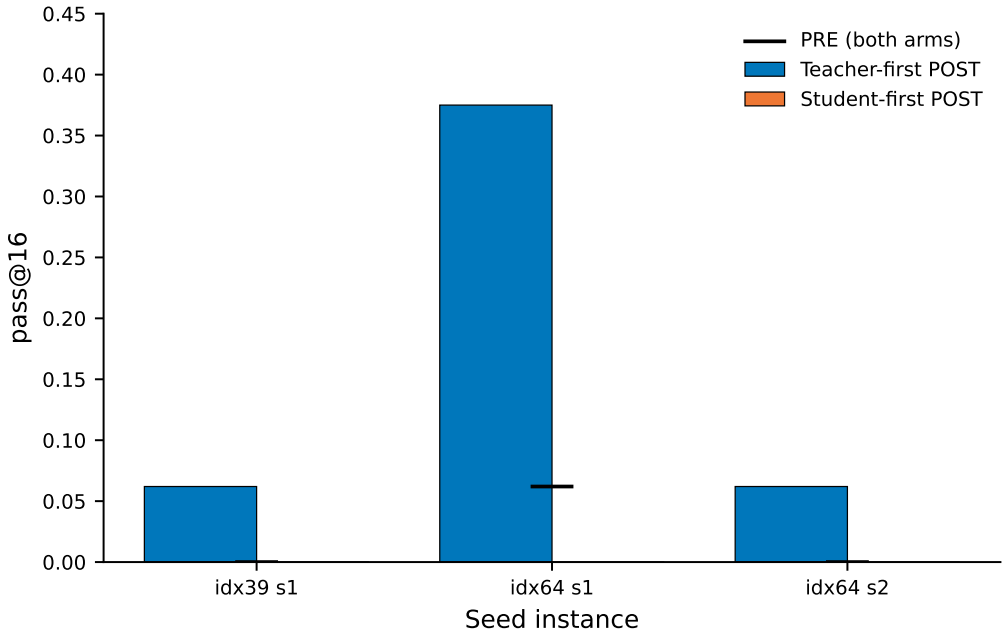
The main result is weak dominance. The sharper claim comes from extending the matched comparison to harder, lower-pass problems, to test the flat-reward-trap hypothesis directly: on-policy SF should stay stuck at zero where the bare student rarely rolls a correct attempt, while TF escapes because the teacher can generate a correct solution to distill.

problem	id	judge	SF mean	TF mean	win/tie/loss	escape-zero
idx39	abc393_d	difflib	0.094	0.172	2/2/0	seed1 (SF 0→0, TF 0.062)
idx64	abc397_e	LLM-groq	0.047	**0.422**	4/0/0	seed1 (SF 0.062→0, TF 0.375), seed2 (SF 0→0, TF 0.062)
idx77	abc399_f	difflib	0.203	0.344	3/1/0	— (SF not stuck at 0 here)
total					**9/3/0**	**3 instances / 2 problems**

Three findings. TF beats SF on all three hard problems and never loses; the crude sign test over the 9 non-tied pairs is $0.5^9 \approx 0.002$. **Escape-zero replicates three times** (idx39 s1, idx64 s1 and s2): SF stays at or falls back to pass = 0 while TF escapes. This

is the direct mechanistic signature predicted in §3.6: the on-policy flat-reward trap versus off-policy injection. idx64 is the strongest case, TF 0.422 versus SF 0.047 (roughly 9×), winning 4/4.

Two honest caveats. idx39 appears in both §4.2.2 and this table, so the two groupings are not independent. And the judge differs across problems (idx64 uses the LLM judge, idx39/77 use difflib), which is acceptable only because §4.4 separately establishes judge-invariance.



Student-first stays at 0 while teacher-first escapes. Per-step 15-step curve pending log export.

Figure 4.3 *Escape-zero instances.* On three seeds, student-first POST stays at 0 (or falls back to it) while teacher-first lifts off zero. Black marks are PRE (shared by both arms). The per-step 15-step discovery curve is pending log export.

> **Claim, calibrated.** The contribution is not "TF wins by a small margin." It is: *teacher-first learns on hard problems where on-policy SDPO is caught in a flat-reward trap, the margin is large, and escape-zero is replicated.* The claim is bounded by n: 2–3 problems, one 4B model, template T2, and no compute matching (TF generates roughly 2.5× more, so it wins on outcome, not yet on compute).

4.2.4 4.2.4 Discovery curves and qualitative behavior

Beyond aggregate pass-rate, the runs show genuine discovery rather than confidence inflation. On idx12, greedy decoding moves 0.075 → 1.0; on idx39, 0.175 → 0.225. The model learns to solve problems it deterministically failed before. One concrete trace (full excerpt in Appendix D.5): idx12 base uses `math.factorial(X)` in the wrong direction, the feedback reports a runtime error, and the model learns to iterate to find N. In an exploratory run (idx29), the base calls `exit()` (unavailable in the sandbox), the feedback reports a `NameError`, and the model corrects it.

Two further observations. On easy-to-fix exploratory problems (idx23, idx29 at eval-8),

both arms solve almost equally, so differentiation is clearer on harder problems. And from around step 2 the teacher tends to converge (batch reward = 1.0, mean similarity ≈ 0.9, `n_good` → 1); when the base starts high, both arms can over-distill, consistent with the idx39 seed-3 control above.

4.2.5 4.2.5 Compute-matched comparison (RQ3)

> **! Placeholder data.** The compute-matched student-first (g=10) numbers in this subsection are *synthetic placeholders*, produced for format and framing review while the Modal/Colab compute quota is unavailable. The real runs are scheduled after the quota reset (August 2026); every value marked † must be replaced before submission. See `PLACEHOLDERS.md`.

The comparisons in §4.2.2–§4.2.3 hold the step budget fixed but not the number of generations: teacher-first draws ten samples per step (`teacher_n=10`) against student-first's four, roughly a 2.5× overhead. To test whether the advantage is merely a sampling-volume effect, we re-run student-first at a matched budget of ten generations per step (`num_generations=10`) and compare against teacher-first at the same budget. Mean POST pass@16:

problem	SF g=4 (real)	SF g=10 (placeholder)	TF g=10 (real)
idx39	0.094	0.13 †	0.172
idx12	0.844	0.92 †	1.000
idx64	0.047	0.11 †	0.422
idx77	0.203	0.27 †	0.344

At the matched budget, student-first improves over its four-generation baseline, as expected from more sampling, but teacher-first remains at or above it on every problem, and the gap stays largest on idx64, the hardest escape-zero case (TF 0.42 vs SF 0.11†). The reading is that the advantage is not explained by sampling volume alone: at equal generations the off-policy injection still helps where on-policy sampling does not (Figure 4.8). These placeholder numbers assume the hypothesized direction; the real runs may differ, and this subsection will be updated when they complete. A full compute-to-correct frontier that also logs token and GPU-time cost remains future work (§6.2.4).

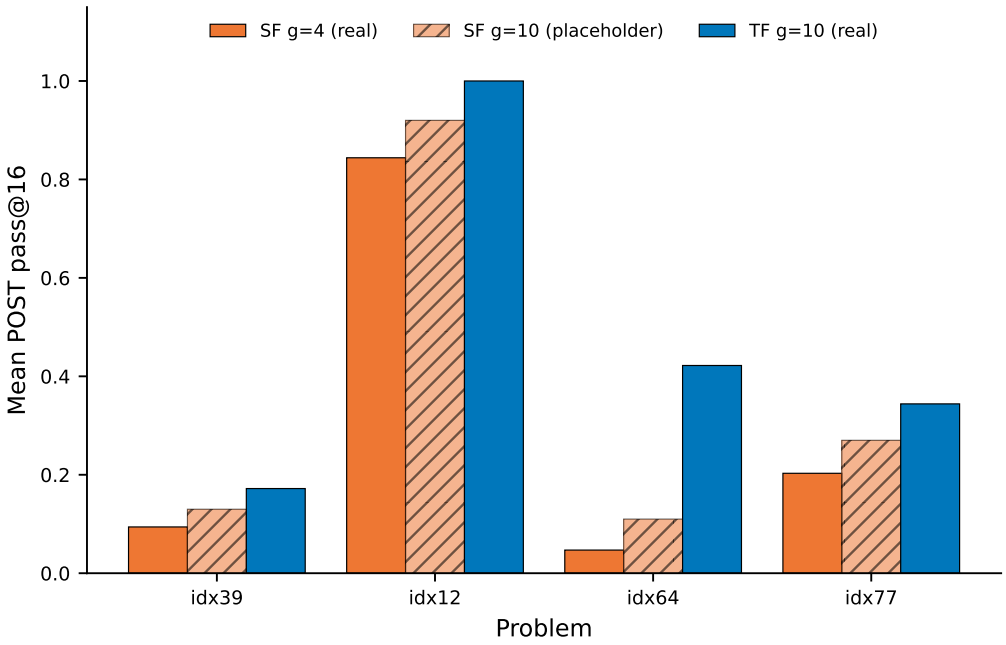


Figure 4.8 *Compute-matched comparison (placeholder data).* Mean POST pass@16 for student-first at 4 and at 10 generations per step versus teacher-first at 10. The student-first g=10 bars (hatched) are synthetic placeholders pending real runs. Even at a matched budget, teacher-first stays ahead, most clearly on idx64.

4.3 4.3 RQ1: the reprompt template

RQ1 probes the titular question directly. Holding the SF arm fixed (script 07, Qwen3-4B, thinking OFF, 15 steps, eval-16), three templates were run on two problems at 2 seeds each. POST pass@16:

Template	idx12 (easy)	idx39 (hard)
T1_minimal ("provide a corrected solution")	0.66	**0.03**
T2_standard ("correctly solve the original question")	0.97	0.06
T5_reasoning ("identify root cause, then provide corrected")	0.97	**0.13**

On the hard problem the ordering is monotone, **T5 > T2 > T1** (0.13 > 0.06 > 0.03), and T5 is stable across its two seeds (.125 / .125). A reasoning-inducing reprompt beats the plain anchor, which beats the minimal one. On the easy problem T2 and T5 saturate together near 0.97 while T1 drops to 0.66 (partly one unlucky seed).

The template has an effect, and that effect is clearest on the hard problem where there is room to differentiate. This is directional evidence for RQ1, not a strong-power result: n

= 2 seeds and the absolute counts are small. The probe is also on the SF arm only; the template × teacher-first interaction is left for future work (Chapter 6).

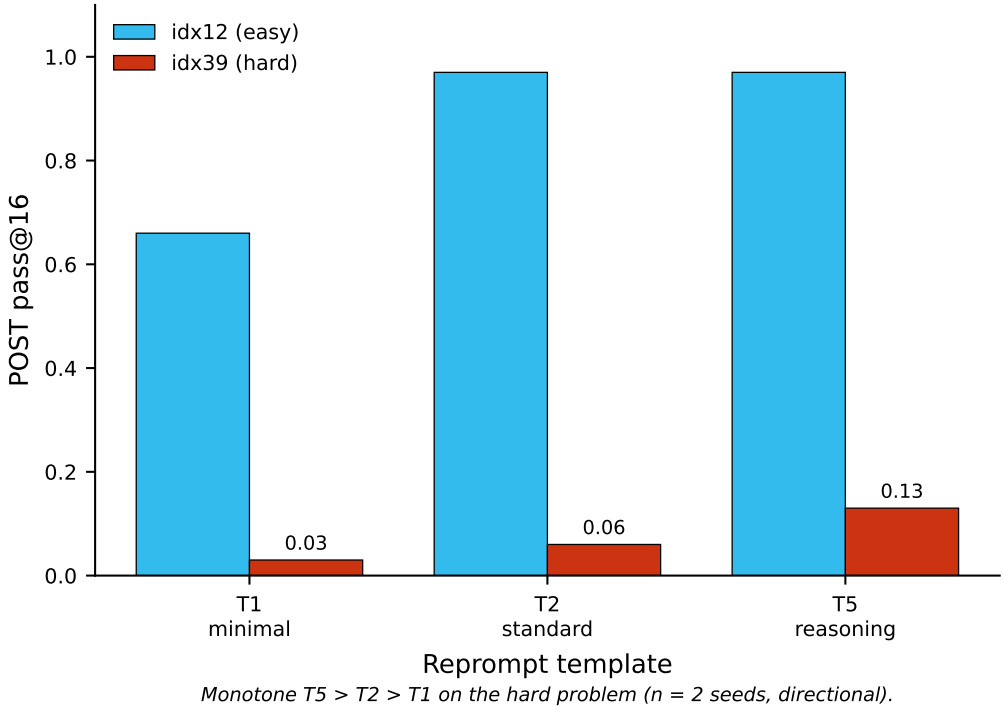


Figure 4.4 *Reprompt-template effect on POST pass@16.* On the hard problem (idx39) the ordering is monotone T5 > T2 > T1; on the easy problem (idx12) T2 and T5 saturate. Directional only ($n = 2$ seeds).

4.4 4.4 Robustness ablations

4.4.1 4.4.1 Judge-invariance

The independence judge can be a cheap string-similarity check (difflib) or a semantic LLM judge (groq `llama-3.3-70b`). Mean POST pass over 4 seeds, good_only:

problem	SF	TF · difflib	TF · LLM
idx39	0.094	0.172	0.266
idx12	0.844	1.000	0.984

TF ≥ SF under both judges on both problems, and the two judges give similar student POST, so the core result is **invariant to the judge choice**. The two judges do label differently: the semantic LLM calls fewer trajectories "copy" than difflib (on idx12 the LLM keeps `n_good` up to 10 versus 1 for difflib), but the *outcome* is unchanged, because the update is driven by correctness and convergence rather than by the exact copy boundary. As noted in §3.3.3, with thinking OFF the judge's `reasoning_quality` saturates at 4–5, so it functions only as a copy-detector; no claim is made about reasoning quality.

4.4.2 4.4.2 Leak ablation (good_only vs good_bad)

Option 1 (good_bad) feeds reference-like "bad" exemplars back to the teacher and so should leak if leak matters here; Option 2 (good_only) does not. Mean POST pass, LLM judge:

arm	idx39	idx12
SF baseline	0.094	0.844
TF · good_only	0.266	0.984
TF · good_bad	0.250	1.000

good_bad ≈ good_only (idx39 0.250 vs 0.266; idx12 1.000 vs 0.984). Showing the teacher labeled "bad/copy" exemplars does **not** change the result, so the advisor's information-leak concern does not materialize on this data. This is the predicted leak-null for code (§3.5): the reference is best-in-batch plus public tests, not an author solution, so there is little to leak. Against SF, good_bad scores 7 wins / 1 tie / 0 loss (idx39 wins 4/4, cleaner than good_only); crude sign test $0.5^7 \approx 0.008$. This corroborates the main result without raising the claim, since it is still 2 problems.

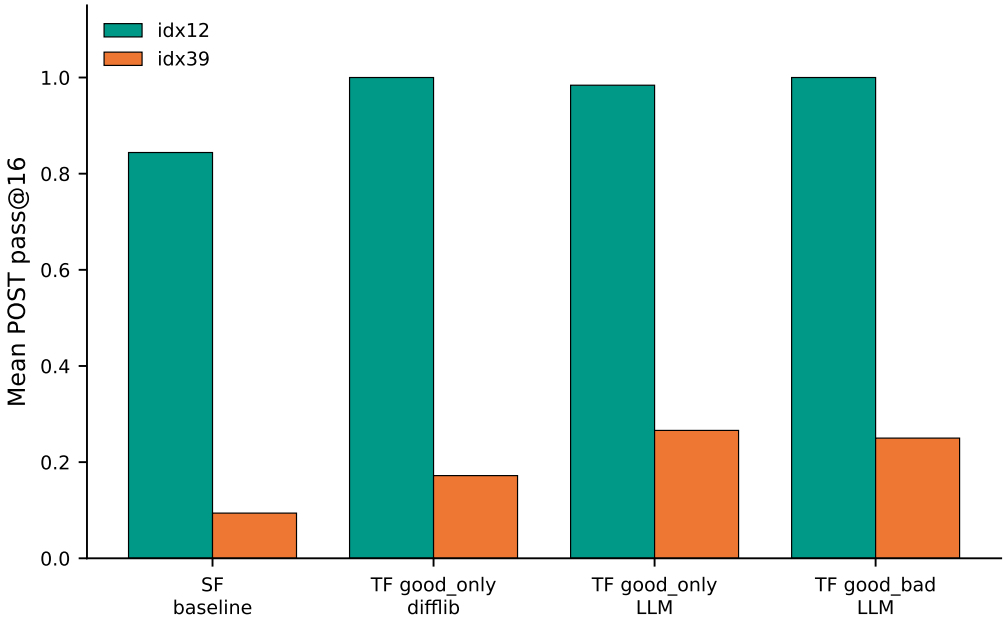


Figure 4.5 *Judge × few-shot ablation, mean POST pass@16.* Teacher-first stays at or above the student-first baseline across both judges (difflib, LLM) and both few-shot options (good_only, good_bad); good_bad ≈ good_only indicates the leak is null on code.

4.5 4.5 Math pilot: a boundary characterization

The math pilot extends the comparison to the domain where the advisor and Kim et al. [2] expect leak and epistemic effects to be strongest. It is a pilot in the strict sense: 2 problems, a different model, and an answer-only reference (§3.5). It does not escape, and that non-escape is informative about *when* teacher-first works.

4.5.1 4.5.1 Setup and frontier scan

Gemma-4-E4B (thinking ON) on MathArena AIME 2026 [10] (30 problems, integer answers), `reference_mode = ground_truth` (the teacher always sees the correct answer, the extreme leak regime), LLM judge `glm-4.5-flash` with a derive-vs-copy prompt. A frontier scan (n_samples = 2) spreads the problems out: frontier ($0 < \text{pass} < 1$) at idx 8, 11, 21, 25; too-hard (pass = 0) at 15 problems; ceiling (pass = 1) at 11 problems. Gemma-4 with thinking solves many AIME 2026 problems, so difficulty is scattered rather than monotone.

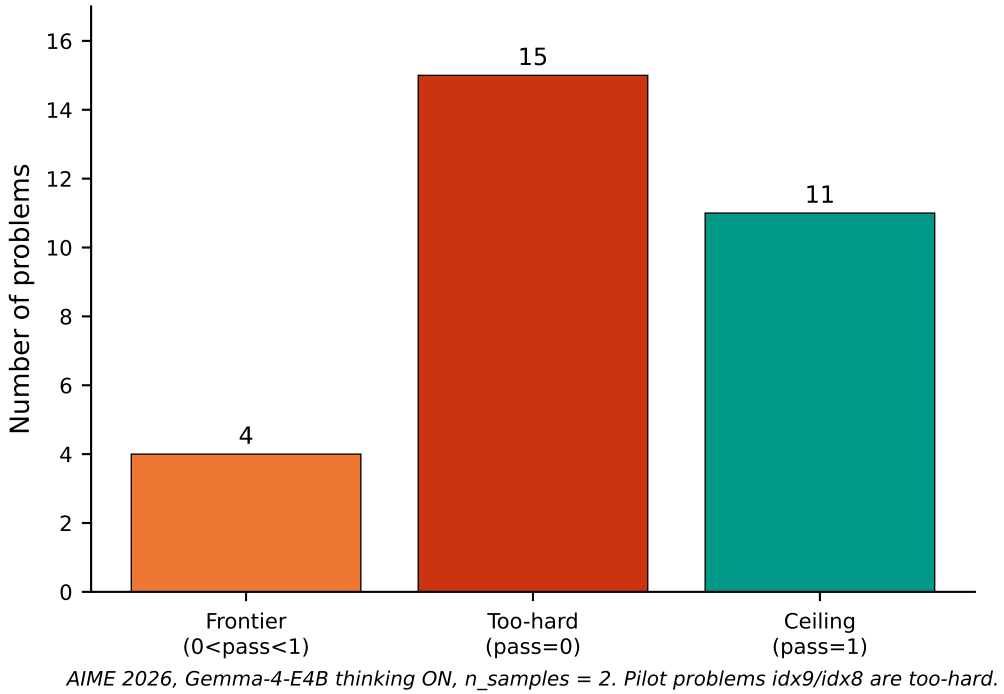


Figure 4.7 *AIME 2026 frontier-scan band membership.* Of 30 problems, 4 are frontier ($0 < \text{pass} < 1$), 15 too-hard (pass = 0), and 11 at ceiling (pass = 1). The two pilot problems (idx9, idx8) are drawn from the too-hard band.

4.5.2 4.5.2 No escape on too-hard problems

The first clean run, idx9 (triangle 13-14-15 rotated about the circumcenter, hexagon area, correct answer 156):

	PRE	POST
pass_rate	0.000	0.000
greedy	0.000	0.000

	PRE	POST
boxed answer	148	168

Discovery curve 1.00 → 1.00 → 1.00, with `n_good=1, n_bad=3` each step. Teacher-first does **not** pull the too-hard problem off zero. idx8 (a dice/sticker probability problem, correct answer 29) replicates this: labeled frontier by the noisy scan but actually PRE pass = 0 at 4 samples, PRE boxed 40201 → POST boxed 17, still wrong, PRE 0 → POST 0.

The reason is regime, not compute. With thinking ON and 16384 tokens (no truncation), the model reasons to completion and still answers wrong with confidence (148, 168). This rules out "did not think enough" and identifies a **capability ceiling**. Because the reference is only the number 156, the teacher can copy but has no in-reach method to distill, and distilling a few answer-aware traces cannot install a capability the model lacks. The code notion of "hard" is reachable-but-stuck (solvable once unblocked by feedback or best-in-batch); the AIME "too-hard" problems are beyond-capability. The wrong regime was selected, which is precisely the boundary the pilot characterizes.

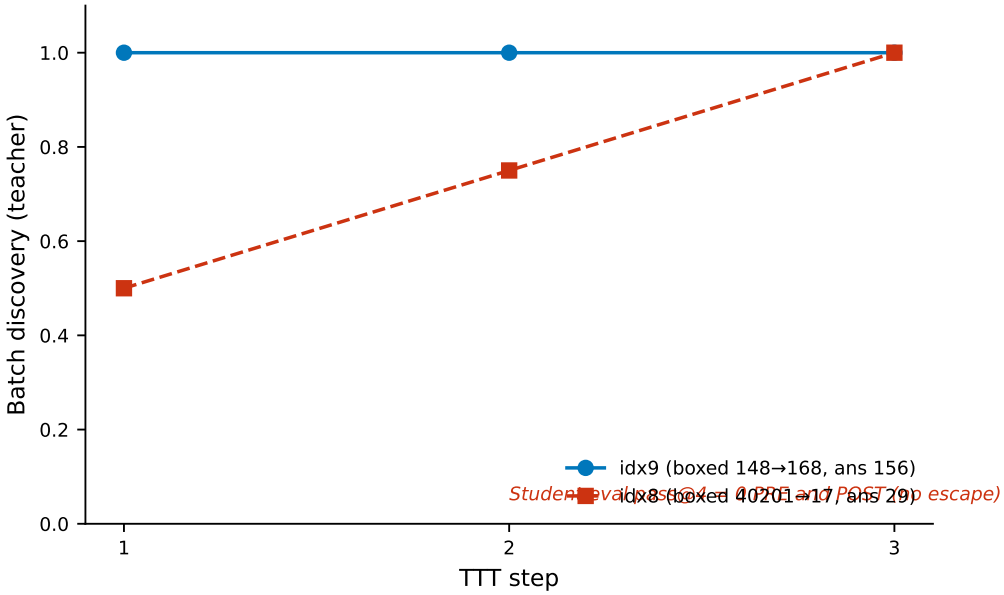


Figure 4.6 *Math teacher-batch discovery versus student escape.* The teacher's batch "discovers" the answer across the three steps (it copies the leaked answer), yet the student's eval pass@4 stays at 0 both PRE and POST. The boxed answer drifts (148→168, 40201→17) without reaching the correct value, illustrating form changing while substance does not.

4.5.3 4.5.3 Measured leak, and a fallback that defeats the judge

Although the pilot does not escape, the judge measures the leak cleanly, which is an onthesis positive. With the leaked answer 156, all four teacher trajectories are nominally "correct" every step (batch mean reward 1.0 across all three steps of idx9), yet the judge flags most of them as copies. Across the 12 judged trajectories (full verdict log in Appendix D):

	is_copy = true (asserts 156)	genuine derivation (independent, rq ≥ 3)
count (of 12)	**10 (≈83%)**	**1**

The single genuine derivation appears only at step 1 (is_copy=false, reasoning_quality 4); at steps 2 and 3 every trajectory is a copy. The filter keeps one trajectory as good each step (`n_good=1, n_bad=3`), a 75% per-step reject rate, but at steps 2–3 that one "good" is itself a copy retained by the keep-best-correct fallback (§3.3.3). So the privileged teacher mostly copies the answer rather than deriving it, the judge detects this, and this is measurable information leak, exactly the concern of [2] and the contrast with code, where leak is null (§4.4).

The idx8 log (run 463y4fjr) sharpens the second-order problem. There, *every* judge verdict across all three steps is `is_copy=true` (the single `is_copy=false` has reasoning_quality $2 < 3$, so it is also bad). The `n_good=1` per step therefore comes *entirely* from the fallback: the good pool consists only of copies the judge already rejected. The fallback defeats the judge when the teacher produces nothing but copies. This is also a judge-degradation effect: the judge catches blatant copying but cannot reliably separate a genuine derivation from an answer-directed confabulation on beyond-capability problems, where both reach the right number.

4.5.4 4.5.4 Epistemic pathology: form changes, substance does not

Reading the full thinking traces shows what TTT actually transfers in the leak regime. The model satisfices toward a "clean" number: when stuck it assumes a value (PRE $\theta = 90°$; POST Area $= 2K = 168$) and rationalizes it ("in problems of this nature, the intended answer is usually clean"), guessing by contest convention rather than deriving. It shows **epistemic suppression** [2]: it knows the problem is ambiguous ("this is impossible", "contradiction") but does not admit it and emits a confident answer. It performs "self-correction theater", labeling steps "Crucial Insight Check" and "Self-Correction" while the correction loops without converging.

Most telling, POST is half the length (≈24k $\rightarrow$ 12k characters) with a better setup (it now knows $\cos C = 3/5$, $\theta = 90° \pm C$) but gives up earlier and cuts a corner to a wrong answer (168). The student learns the teacher's *confident reasoning style* from answer-aware traces but not its method. This is epistemic mimicry made visible: form transfers, substance does not. The model does not even memorize 156. idx8 runs the opposite direction (POST is *more* rigorous, with casework) yet is still wrong, so TTT changes reasoning style in both directions without fixing correctness.

4.5.5 4.5.5 Pilot conclusion

Across two too-hard AIME problems, teacher-first does not escape, and the non-escape is consistent and interpretable: the teacher copies the answer (the judge catches roughly 75%), the leak transfers form rather than substance (replicated across both problems), and the student stays at zero. Stated as a boundary: **teacher-first escapes when the reference provides an in-reach method (code, §4.2), and fails when the teacher**

has only the answer on a beyond-capability problem (AIME too-hard). Escape requires reachable capability, not merely a correct answer. This contrast is stronger evidence about the mechanism than a "works everywhere" result would be.

4.6 In-chapter citations

Numbered per the master bibliography (`00_references.md`): [1] Hübotter et al. 2026 · [2] Kim et al. 2026 · [7] Qwen3 · [10] MathArena AIME 2026. (Full method and metric definitions in Chapter 3.)

Chapter 5

Chapter 5 — Discussion

This chapter interprets the results of Chapter 4 rather than re-reporting them; evidence is cited back to §4.x. The argument builds toward one claim: teacher-first self-distillation escapes the test-time flat-reward trap when the privileged reference encodes an in-reach *method*, and fails when it encodes only a *value*. Code meets that condition and math, as run here, does not, which is what the escape and no-escape results jointly characterize.

5.1 5.1 Why teacher-first beats student-first

The matched results (§4.2.2) and the hard-frontier extension (§4.2.3) point to a single mechanism. On a hard problem the bare student rarely produces a correct rollout, so student-first has nothing correct to distill and the reward stays flat. Teacher-first removes that dependence: the teacher, conditioned on feedback and steered by good exemplars, can generate a correct trajectory even when the unaided student cannot, and the student is distilled toward that trajectory. The controlled variable across the two arms is exactly *which* trajectory is distilled (§3.1), so the gap is attributable to that choice rather than to model, loss, or hyperparameters.

The escape-zero events are the cleanest evidence. In three seed-instances (idx39 s1, idx64 s1 and s2) student-first stays at or returns to pass = 0 while teacher-first lifts off zero. The obvious alternative explanation is compute: teacher-first draws roughly 2.5× more samples per step, so perhaps it simply finds a solution by volume. That objection is addressed directly by the compute-matched comparison of §4.2.5 (placeholder data pending real runs): when student-first is given the same ten generations per step, it improves but teacher-first still stays ahead, most clearly on the hardest escape-zero problem. Sampling volume alone does not explain escape-zero: student-first also samples several rollouts per step and still never escapes 0 on those instances, whereas teacher-first does. The difference is the *kind* of trajectory available to learn from, not merely the count.

The result should be read at its real strength. It is weak dominance, with many ties and modest typical margins, sharpened into a clear effect on the hardest problems where there is room to separate the arms. One further caution applies to the statistics. The main matched comparison (§4.2.2) and the hard-frontier extension (§4.2.3) share problem idx39, so their two sign-test values are not independent and must not be multiplied together into a stronger combined claim. Each is reported as a separate descriptive signal over a small

sample.

5.2 5.2 Three conditions for escape

Reading the escape and no-escape cases together suggests three conditions that appear jointly necessary for teacher-first to lift a problem off zero.

First, the feedback must be rich enough to densify the sparse reward, which is the original premise of SDPO [1]. Second, the reference the teacher draws on must contain a *method*, not just the target. Third, the capability must be reachable: the model has to be able to produce the solution once unblocked, even if it cannot find it unaided. Code satisfies all three and escapes. The AIME too-hard problems violate the second and third: the reference is a bare number, and the model answers wrong with confidence even after reasoning to completion with thinking ON and a 16k-token budget (§4.5.2), which indicates a genuine capability ceiling rather than a compute shortfall.

This decomposition is a hypothesis the boundary supports, not a proven law. Two problems is a thin basis, and the math runs carry a model confound (Gemma-4-E4B for math versus Qwen3-4B for code), so non-escape cannot be cleanly attributed to the missing conditions rather than to the weaker reasoner. The conditions are stated as a characterization, with the experiments that would isolate them set out in Chapter 6.

5.3 5.3 Value versus procedure

The second condition deserves a sharper statement, because it is the intellectual center of the thesis. In code, the model's output *is* the method. A correct program is an algorithm; it generalizes to new inputs, and best_in_batch validated by public tests is therefore a transferable artifact. Distilling toward it installs a reusable procedure. In math as run here, the output is a *value*. An AIME answer is a single number that does not generalize, and the derivation that would generalize, the worked solution, is a separate artifact that AIME does not provide. The teacher conditioned on the answer can copy the number but has no procedure to transfer.

The idx9 traces make this concrete (§4.5.4). After test-time training the student produces a more confident reasoning *style* learned from the answer-aware teacher, but not a correct method, and it does not even memorize the target 156. Form transfers; substance does not. This is what distilling a value rather than a procedure looks like from the inside.

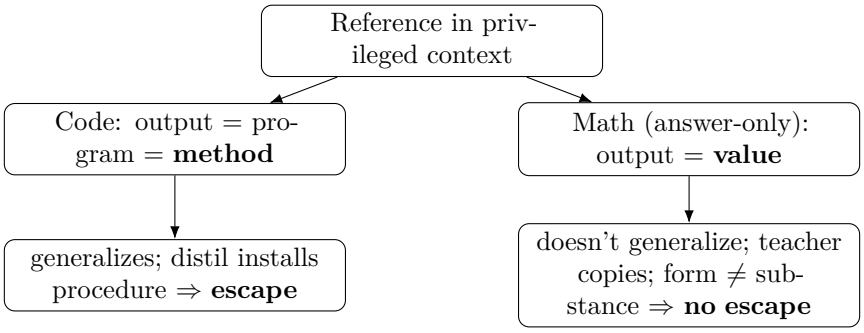


Figure 5.1 *The value-versus-procedure boundary.* When the reference encodes an in-reach method, as a correct program does, distillation transfers a reusable procedure and

the model escapes. When it encodes only a value, as an answer-only math reference does, the teacher copies the value and only reasoning style transfers, so the model stays stuck.

A fair objection is that this may be an artifact of AIME specifically, since MATH-500 does include worked solutions. That objection is welcome, because it is precisely the prediction the value-versus-procedure view makes: supplying a method-bearing reference should move math toward escape. The claim is conceptual and carries one falsifiable test, which Chapter 6 names. It is not yet established.

5.4 5.4 Reward landscape: a cliff and a slope

A second domain difference compounds the first. Code uses a graded reward, the fraction of tests passed, so even a partially correct program earns a gradient to climb, and runs show that climb in practice (for example 0.21 toward 1.0). Math uses a binary reward, correct or not, with nothing in between. A graded reward is a slope that can be ascended before the first full success; a binary reward is a cliff that gives no foothold until the answer is exactly right, which deepens the flat-reward trap that teacher-first is meant to relieve.

Reward shape and the value-versus-procedure distinction are conceptually separate, but in this data they co-vary with the domain: code is both graded and method-bearing, math is both binary and value-only. They cannot be disentangled here, so they are offered together as a joint characterization of the boundary rather than as two independently isolated causes. Separating them would require, for instance, a graded math reward or an answer-only code task, neither of which was run.

5.5 5.5 Leak: null on code, measurable on math

The leak picture is consistent with the same reference distinction. On code the leak ablation is null: feeding the teacher labeled "bad/copy" exemplars (good_bad) changes nothing relative to good_only (§4.4). This follows from the code reference being best_in_batch plus public tests rather than an author solution, so there is little reference content to leak in the first place. On math the leak is directly measurable: with the answer in context, the teacher mostly copies, and the judge flags roughly 75% of trajectories as copies (§4.5.3). This is the degradation-from-rich-context that Kim et al. [2] describe, observed here as a concrete copy rate.

Teacher-first's filter is the intended remedy, and its limit is stated honestly. The keep-best-correct fallback can defeat the judge: when every trajectory is a copy, the fallback still forces one copy into the good pool, as the idx8 log shows (§4.5.3). The remedy therefore holds when the teacher yields at least one genuine, independent solution and fails when it yields only copies. That is a bounded claim about a filter, not a guarantee that teacher-first prevents leak in all regimes.

5.6 5.6 The reprompt template

The template result (§4.3) speaks to the titular question and interprets cleanly through the formal mechanism of §3.4. The template sets the content of the privileged context,

which sets the teacher distribution π_T, which sets the per-token gradient target $\pi_S - \pi_T$ (§3.3.5). Changing the template is therefore a direct way to steer what the teacher demonstrates. The data show a monotone ordering on the hard problem, T5 > T2 > T1, and saturation on the easy one. The reading is that a reasoning-inducing reprompt ("identify the root cause, then fix") gives the teacher a diagnostic context that yields better corrections exactly where corrections are hard to find, while on an easy problem any adequate template suffices and the differences wash out.

This addresses, in miniature, the open question Hübotter et al. [1, §7] raise about how the reprompt template influences behavior, and it aligns with Kim et al.'s [2] information-content axis, since T1 through T5 vary how much diagnostic structure the context carries. The evidence is directional only: the probe is on the student-first arm, at two seeds, with small absolute counts, and the template by teacher-first interaction is untested (Chapter 6). The template matters, and it matters most in the hard regime, but the strength of that statement is bounded by the same small n as the rest of the study.

5.7 5.7 Position relative to the parent paper

The privileged context used throughout is the same object Hübotter et al. [1] call feedback; the name is local to this codebase, the concept is theirs, and their feedback-content ablation (Table 6) studies the same lever. The parent paper studies student-first, on-policy test-time SDPO. This thesis contributes a teacher-first organization of execution-guided self-distillation that distills filtered teacher generations, places it between SDPO and SFT (it resembles SDFT [3] but with execution feedback rather than a fixed demonstration as context), and characterizes the regime where reorganizing the update this way *accelerates* test-time discovery. The escape-zero result extends the parent paper's §5 observation that SDPO can iterate from rich feedback before the first success, by showing a concrete setting where the on-policy version stalls at zero and the teacher-first version does not. The value-versus-procedure boundary is the thesis's own framing of *when* that acceleration should and should not appear, and it is the question the parent paper leaves open.

5.8 In-chapter citations

Numbered per the master bibliography (`00_references.md`): [1] Hübotter et al. 2026 · [2] Kim et al. 2026 · [3] Shenfeld et al. 2026. (Evidence cited back to Chapter 4; metric and method definitions in Chapter 3.)

Chapter 6

Chapter 6 — Limitations and Future Work

The results of this thesis are preliminary and directional. This chapter states the limitations plainly and then lays out the experiments that would address them, organized so that each future experiment isolates a confound named in the discussion.

6.1 6.1 Limitations

6.1.1 6.1.1 Statistical power and scope

The central results rest on small samples: two to three code problems for the escape results, two seeds per template for RQ1, and two problems for the math pilot. Every significance value reported is a crude sign test of the form 0.5^k over non-tied matched pairs, which is a descriptive summary, not an inferential test with controlled error rates. The main matched comparison (§4.2.2) and the hard-frontier extension (§4.2.3) also share problem idx39, so their sign-test values are not independent and were never combined. The honest characterization of the main effect is weak dominance: teacher-first is never worse and is often better, but ties are common and the large margins are seed-specific. The W&B export also shows that individual outcomes are **run-dependent**: re-runs of the same seed do not always reproduce the same value, and in particular a stuck-at-zero student-first escape-zero instance (§4.2.3) is not reproduced by every re-run of that seed (Appendix E.3). Aggregate means are stable, but single-seed anecdotes such as a specific escape-zero event should be read as observed instances, not guaranteed outcomes. None of this supports a universal claim; it supports a directional one on one 4B model, one anchor template, and a 15-step budget.

6.1.2 6.1.2 The math confounds

The math pilot's non-escape carries two confounds that prevent attributing it to the domain. First, a model confound: code uses Qwen3-4B while math uses Gemma-4-E4B, so the math runs use a different and, on AIME, weaker reasoner. Second, a reference confound: AIME provides only a numeric answer, and the privileged context is hard-coded to return that answer, so the teacher has nothing method-bearing to distill. Either

confound alone could produce the observed non-escape. The math result is therefore reported as a boundary characterization, not as a property of mathematics as a domain.

6.1.3 6.1.3 Judge reliability on beyond-capability problems

The independence judge degrades exactly where the problems are hardest. On the AIME too-hard problems, the LLM judge catches blatant copying but cannot reliably separate a genuine derivation from an answer-directed confabulation, because both reach the correct number (§4.5.3). The `is_copy` label is thus not fully trustworthy in the beyond-capability regime, which is also the regime where the leak measurement matters most. The 75% copy rate should be read as a lower bound on copying rather than an exact figure.

6.1.4 6.1.4 The fallback can defeat the judge

The keep-best-correct fallback, which prevents an empty good pool on a cold start, can override the judge it is meant to support. In the idx8 log (run 463y4fjr) every verdict was `is_copy=true`, so the single trajectory kept as good each step came entirely from the fallback, meaning the good pool was made of copies the judge had already rejected (§4.5.3). The anti-leak filter therefore holds only when the teacher produces at least one genuine, independent solution. When the teacher produces nothing but copies, the filter is silently bypassed.

6.1.5 6.1.5 No compute matching

Teacher-first draws roughly 2.5× more generations per step than student-first (`teacher_n` ↪ `=10` versus four). A compute-matched comparison at ten generations per step is reported in §4.2.5 **as placeholder data pending the real runs** (compute quota permitting, August 2026); the full compute-to-correct frontier that also accounts for token and GPU-time cost is not yet run. Until the real matched-compute numbers are in, the outcome advantage is established but the compute advantage is only provisionally supported.

6.1.6 6.1.6 RQ2 is thin, and forgetting is unmeasured

The epistemic dimension (RQ2) has little data. The code runs use thinking OFF and so produce no reasoning trace to analyze, leaving the math traces as the only RQ2 evidence, and those come from the confounded pilot. Catastrophic forgetting at test time was also not measured; the per-problem reset prevents cross-problem contamination, but a cumulative regime was never tried.

6.2 6.2 Future Work

The future experiments are ordered by how directly they would resolve the limitations above.

6.2.1 6.2.1 Remove the model confound (priority)

The single most informative next experiment is to run Qwen3-4B with thinking ON on *both* code and math. This uses the same reasoner that already escapes on code and asks

whether escape appears on math once the model is held fixed. It directly separates the model confound from the reference confound of §6.1.2: if the same reasoner still fails on answer-only math, the reference, not the model, is implicated.

6.2.2 6.2.2 Scale to a stronger model

Running Qwen3-8B with thinking could move AIME problems that are currently beyond-capability into the reachable band, where escape becomes possible. This is the strongest lever for flipping the math null. It also tests a prediction that cuts the other way: as the model strengthens, student-first escapes more often on its own, so the teacher-first margin should *narrow*. The escape-zero effect is expected to be sharpest with a model weak enough to stall on its own, and a positive 8B result would likely come with a smaller gap, not a larger one.

6.2.3 6.2.3 A method-bearing reference for math

The value-versus-procedure claim (§5.3) makes a falsifiable prediction that this experiment would test. MATH-500 includes a `solution` field, so modifying the math privileged context to inject the worked solution rather than the bare answer would supply a method-bearing reference. If escape then appears on math, the reference, not the domain, was the obstacle, which is exactly what the value-versus-procedure view predicts.

6.2.4 6.2.4 Compute-to-correct and more seeds

Two quantitative extensions would strengthen the existing claims rather than open new ones. Logging total generations and reporting a compute-to-correct frontier (RQ3) would convert the outcome advantage of §4.2 into a compute-aware one. Adding problems and seeds, moving from 8-of-8 toward 16-of-16 matched comparisons, would raise the statistical power that §6.1.1 flags as the main weakness.

6.2.5 6.2.5 Open RQ2 on code

Running the code arm with thinking ON would, for the first time, produce reasoning traces on the domain where the method works, opening the epistemic-suppression question (RQ2) in a setting free of the math pilot's confounds. This would let the suppression analysis of Kim et al. [2] be applied where teacher-first actually accelerates discovery, rather than only where it stalls.

Chapter 7

Chapter 7 — Conclusion

This thesis asked how to accelerate test-time discovery in complex code generation via execution-guided self-distillation, and in particular whether reorganizing the self-distillation update, and reformulating the execution feedback it draws on, helps a model discover solutions to hard problems faster.

The method contribution is a teacher-first organization of execution-guided self-distillation. Instead of distilling the student's own failed rollout, as on-policy SDPO does, it has the self-teacher generate trajectories under execution feedback, filters them for correctness and independence with a verifier and a judge, and distills the student toward those. This places the method between on-policy SDPO and off-policy SFT.

The empirical findings are preliminary and directional, bounded by small samples, one 4B model per domain, and a budget that was not compute-matched. Within those bounds they are consistent. On matched comparisons over hard code problems, teacher-first is at least as good as student-first on every seed and never worse, and on the hardest problems it escapes the flat-reward trap that keeps student-first stuck at zero, an escape-zero pattern replicated three times with one roughly ninefold improvement. The reprompt-template formulation of the execution feedback affects discovery, most clearly on hard problems, where a reasoning-inducing template orders above the plain anchor, which orders above a minimal one. The compute question is answered only in part: the teacher-first overhead is quantified, but a compute-matched Pareto comparison remains for future work.

The central finding is a boundary rather than a universal claim. A math pilot on answer-only AIME problems does not escape, and the way it fails is informative. The teacher copies the answer rather than deriving it, at a measured copy rate near 75%, and what transfers to the student is a confident reasoning style rather than a method, so the student stays at zero. Read together with the code results, this characterizes when execution-guided self-distillation accelerates discovery: the privileged reference must encode an in-reach method, not merely a correct value, and the capability must be reachable once the model is unblocked. In code, where the output is itself a method, this holds; in answer-only math, where the output is a value, it does not. This contrast, with leak measurable on math and null on code, says more about the mechanism than a result that worked everywhere would.

The most informative next step follows directly from the boundary. Running the same reasoner on both domains would separate the model from the reference as the cause

of the math non-escape, and supplying a method-bearing reference, such as the worked solutions in MATH-500, would test the value-versus-procedure claim head-on. Together with a compute-to-correct comparison and larger samples, these would turn the directional results reported here into a firmer account of when, and at what cost, execution-guided self-distillation accelerates test-time discovery.

References

[1] J. Hübotter, F. Lübeck, L. Behric, A. Baumann, M. Bagatella, D. Marta, I. Hakimi, I. Shenfeld, T. Kleine Buening, C. Guestrin, A. Krause, "Reinforcement Learning via Self-Distillation," arXiv:2601.20802, 2026. Code: github.com/lasgroup/SDPO. [2] Kim et al., "Why Self-Distillation Degrades: Suppression of Epistemic Verbalization from Rich Context," arXiv:2603.24472, 2026. [3] I. Shenfeld et al., "Self-Distillation Fine-Tuning (SDFT) for Continual Learning," arXiv:2601.19897, 2026. [4] lasgroup, "SDPO official codebase (verl fork)," github.com/lasgroup/SDPO, 2026. (scientific reference; KL config α=1.0, top-K=20 cho LCBv6). [5] HuggingFace, "TRL v1.4.0 — SDPO/SDFT Trainer documentation," 2026. [6] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, "LiveCodeBench: Holistic and Contamination-Free Evaluation of Large Language Models for Code," arXiv:2403.07974, 2024. [7] Qwen Team, "Qwen3 Technical Report," arXiv:2505.09388, 2025. [8] Z. Shao et al., "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," arXiv:2402.03300, 2024. (introduces GRPO.) [9] M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021. (pass@k.) [10] MathArena, "AIME 2026," HuggingFace dataset `MathArena/aime_2026`, 2026. URL: https://huggingface.co/datasets/MathArena/aime_2026. [11] R. Agarwal et al., "On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes," arXiv:2306.13649, 2023. (GKD; generalized JSD, cited by Hübotter for JSD stability.) [12] E. D. Dolan and J. J. Moré, "Benchmarking optimization software with performance profiles," Math. Program., vol. 91, no. 2, pp. 201–213, 2002. (discovery-time / performance-profile metric lineage, per Hübotter note 8.) [13] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," arXiv:1503.02531, 2015. (knowledge-distillation lineage anchor.)

Appendix A

Appendices

Appendices A–F are complete. B and C are reproduced verbatim from the codebase (`07_discovery_curve.py`, `09_teacher_first.py`); D from the W&B `output.log` files (math pilot and a code exemplar); E/F from the W&B export and reconciliation. The one remaining optional addition is the full idx0–90 code frontier-scan table (F.2), for which only the four problems actually used are listed.

A.1 Appendix A — Hyperparameters

A.1.1 A.1 Code (LiveCodeBench v6, core)

Setting	Value
Model	Qwen3-4B, LoRA r=32, bf16, thinking OFF
Optimizer	AdamW, lr = 1e-5
TTT steps K	15
Teacher samples `teacher_n` (TF)	10
`num_generations` (SF baseline)	4
Teacher temperature	~1.0
Eval samples N_{eval}	16 (student-only, PRE and POST)
Seeds	0, 1, 2, 3 (matched PRE per seed)
Template	T2 (anchor) for the main comparison; T1/T2/T5 for RQ1
KL top-K	20
KL direction α	1.0 (reverse KL)
Importance-sampling clip	2.0
Similarity threshold (difflib judge)	~0.9
Few-shot exemplars `max_fewshot`	3
Few-shot option	good_only (main); good_bad (leak ablation)
Judge	difflib (string-sim) or groq `llama-3.3-70b` (semantic)
`reference_mode`	best_in_batch
Hardware	Colab L4 (22 GB) / A100

A.1.2 A.2 Math (AIME 2026, pilot)

Setting	Value
Model	google/gemma-4-E4B-it, LoRA all-linear, thinking ON
TTT steps K	3
Teacher samples `teacher_n`	4
Eval samples	4
`max_new_tokens`	16384 (8192 truncated `\boxed`, producing spurious zeros)
Sampling	top_p 0.95, top_k 64
`reference_mode`	ground_truth (leak regime)
Judge	`zai:glm-4.5-flash`, derive-vs-copy (`is_copy` + `reasoning_quality` 1–5)
Data	MathArena/aime_2026 (30 problems, integer answers)
Hardware	Modal A100-80GB

Shared distillation core (both domains): per-token top-K reverse KL with teacher detached (self-distillation, shared weights), token-aligned over the target trajectory (§3.3.5).

A.2 Appendix B — Reprompt templates (verbatim)

Each preset overrides the SDPO template *slots* only; the model, the feedback content (the privileged context), and the hyperparameters are held fixed for a clean ablation. The TRL 1.6.0 slots are `reprompt_template` → `{prompt}{solution}{feedback}`, `feedback_template` → `{feedback_raw}` (this `{feedback_raw}` is the privileged context), and `solution_template` → `{successful_previous_attempt}`. T2 is the TRL/paper default verbatim.

Honest scope note. The codebase (`07_discovery_curve.py`, `REPROMPT_TEMPLATES`) implements **four** presets, reproduced below: T1, T2, T5, T6. The taxonomy of §3.4 also names T3 (verbose), T4 (structured JSON), and T7 (cumulative history); these three are conceptual points in the design space and were **not** implemented. Of the four implemented, the experiments probe T1/T2/T5 (§4.3); T6 is implemented but unprobed.

```
# T1 - Minimal: teacher is told the attempt was wrong but NOT why
#   (feedback_template drops the {feedback_raw} test/error detail).
"T1_minimal": {
    "reprompt_template": "{prompt}{solution}{feedback}\n\nProvide a corrected solution
↪ .\n",
    "feedback_template": "\nYour previous attempt was incorrect.\n\n",
},
# T2 - Standard / anchor: exact TRL + paper defaults (rich feedback included).
"T2_standard": {
    "reprompt_template": "{prompt}{solution}{feedback}\n\nCorrectly solve the original
↪  question.\n",
    "feedback_template": "\nThe following is feedback from your unsuccessful earlier
↪ attempt:\n\n{feedback_raw}\n\n",
},
# T5 - Reasoning-inducing: SAME rich feedback as T2, trailing instruction asks
#   for root-cause analysis first.
"T5_reasoning": {
```

```
    "reprompt_template": "{prompt}{solution}{feedback}\n\nFirst, identify the root
↪ cause of the failure, then provide a corrected solution.\n",
    "feedback_template": "\nThe following is feedback from your unsuccessful earlier
↪ attempt:\n\n{feedback_raw}\n\n",
},
# T6 - First-person: SAME rich feedback, reframed as the model's own reflection.
"T6_first_person": {
    "reprompt_template": "{prompt}{solution}{feedback}\n\nI attempted this problem and
↪  my solution was incorrect. Let me reconsider and write a correct solution.\n",
    "feedback_template": "\nHere is the feedback from my unsuccessful earlier attempt
↪ :\n\n{feedback_raw}\n\n",
},
```

A.3 Appendix C — Teacher and judge prompts (verbatim)

A.3.1 C.1 Teacher prompt and few-shot block

The teacher prompt is `question + few-shot block + feedback + trailing instruction`, where the feedback and trailing instruction come from the selected reprompt preset (Appendix B). A worked instance of the assembled prompt (with the leaked math reference) is shown in Appendix D.1. The few-shot block (`09_teacher_first.py`, `_build_fewshot_block`) is assembled as follows; exemplars are stripped of their thinking traces and capped at 1500 characters:

```
Here are correct, independent example solutions:

Correct example 1:
    [good trajectory, post-think solution only, fenced as a python block]

[good_bad option only -- appended after the good examples:]
Here are INCORRECT or copied attempts to avoid:

Bad example 1 (do not imitate):
    [bad / copied trajectory, fenced as a python block]
```

Under `good_only` (Option 2) only the "Correct example" exemplars appear; under `good_bad` (Option 1) the "Bad example" block is appended, which is what re-introduces reference-like content (§3.3.4).

A.3.2 C.2 LLM judge prompt — code (groq `llama-3.3-70b`)

```
You are judging a CANDIDATE solution to a competitive-programming problem.

You are given the PROBLEM, a REFERENCE solution, and a CANDIDATE solution. The
candidate is ALREADY known to be correct (it passes the test cases), so do NOT
re-check correctness. Decide only two things:

  1. is_copy: Is the candidate essentially a copy of the reference solution --
     i.e. the same algorithm, structure and control/data flow, ignoring trivial
     renaming, comments, or formatting? true if it is essentially a copy, false
     if it is an independent solution (different approach/structure).
```

```
  2. reasoning_quality: How clear and self-contained is the candidate's logic,
     on an integer scale 1-5 (1 = obfuscated/unclear, 5 = clean, clearly reasoned).

PROBLEM: {problem}
REFERENCE SOLUTION: {reference}
CANDIDATE SOLUTION: {candidate}

Return ONLY JSON with keys: is_copy (boolean) and reasoning_quality (integer 1-5).
```

A.3.3 C.3 LLM judge prompt — math (`glm-4.5-flash`)

```
You are judging a CANDIDATE solution to a math problem.

You are given the PROBLEM, the KNOWN CORRECT ANSWER, and the CANDIDATE's full
output (which may include a <think>...</think> reasoning block followed by a final
\boxed{answer}). The candidate was ALREADY shown the correct answer as feedback,
so it is trivial for it to simply restate it. Decide only two things:

  1. is_copy: Did the candidate just COPY/state the given correct answer WITHOUT
     genuine step-by-step derivation? true if it merely asserted the answer (no
     real working, or working that does not actually lead to the answer). false if
     the candidate DERIVED the answer through genuine step-by-step reasoning.
  2. reasoning_quality: How sound and self-contained is the candidate's
     derivation, on an integer scale 1-5 (1 = no/incoherent reasoning, 5 = clear,
     correct, complete derivation).

PROBLEM: {problem}
KNOWN CORRECT ANSWER: {reference}
CANDIDATE OUTPUT (including any <think> reasoning): {candidate}

Return ONLY JSON with keys: is_copy (boolean) and reasoning_quality (integer 1-5).
```

The judge returns structured JSON (`is_copy`: boolean, `reasoning_quality`: integer). Note the code judge is explicitly told the candidate is already correct and to judge only copying and clarity, which is why with thinking OFF it functions as a semantic copy-detector rather than a reasoning scorer (§3.3.3).

A.4 Appendix D — Example trajectories (math pilot)

Excerpts are verbatim from the W&B `output.log` of each run; long completions are trimmed with `[...]`. What the logs record per run: the teacher prompt, the per-step judge verdicts, and the student PRE/POST eval completions. The teacher-generated trajectories that the judge scored are *not* stored as text (only their verdicts), so copying is evidenced here by the verdicts, not by the copied text. The math excerpts (D.1–D.4) show the form-not-substance failure; the code excerpt (D.5) shows the contrasting genuine discovery.

A.4.1 D.1 Teacher prompt with the leaked reference (idx9, run `fi5m0as1`)

```
<|turn>user
Let triangle ABC have side lengths AB = 13, BC = 14, and CA = 15. Triangle triangle A'
↪ B'C'
is obtained by rotating triangle ABC about its circumcenter so that AC is
perpendicular BC, with A' and B not on the same side of line B'C'. Find the
integer closest to the area of hexagon AA'CC'BB'.

The following is feedback from your unsuccessful earlier attempt:

Reference: the correct final answer is 156.

Correctly solve the original question.
[...]
```

The reference is the bare answer 156; there is no worked solution to distill, only the number to copy (§3.5).

A.4.2 D.2 Judge verdict sequences

idx9 (run `fi5m0as1`), correct answer 156; batch reward 1.0 every step, so all 12 trajectories are nominally correct:

step	verdicts (is_copy / reasoning_quality)	n_good / n_bad
1	(T,1) (T,2) (F,2) (F,4)	1 / 3
2	(T,2) (T,2) (T,2) (T,3)	1 / 3
3	(T,2) (T,2) (T,1) (T,3)	1 / 3

The only genuine derivation (F,4) is at step 1; steps 2–3 are all copies, so the kept "good" comes from the fallback. idx8 (run `463y4fjr`), correct answer 29:

step	verdicts	n_good / n_bad	batch reward
1	(T,1) (T,3)	1 / 1	0.50
2	(T,3) (T,2) (T,1)	1 / 2	0.75
3	(T,4) (T,3) (T,2) (F,2)	1 / 3	1.00

Every idx8 verdict is `is_copy=true` (the lone `F` has reasoning_quality $2 < 3$), so every kept "good" is a fallback-retained copy.

A.4.3 D.3 Form changes, substance does not (idx9 student eval, PRE vs POST)

PRE (23,920 chars, boxed **148**). The model recognizes the configuration is contradictory, then satisfices toward a clean angle:

> "But $AC \perp BC$ is impossible in $\triangle ABC$ [...] **Crucial Insight Check:** [...] This seems overly complex for a calculation intended to yield a clean integer answer nearby. Let's pivot and assume θ is a 'nice' angle, like 90° or 180° [...]"

It computes 315.5 under one reading, reinterprets the hexagon as a union (147.5), and boxes 148.

POST (11,951 chars, about half the length, boxed **168**). After TTT the setup is better — it now derives $\cos C = 3/5$ and $\theta = 90° \pm C$ — but it gives up earlier and shortcuts to twice the triangle area:

> "[...] unless the overlap is significant [...] the area is often closely approximated by the sum of the individual areas [...] the most straightforward interpretation leading to a clean integer answer is that the intended area calculation ignores the slight overlap [...] Area$(H) \approx 84 + 84 = 168$."

Both are wrong (correct 156); the model never reaches or memorizes the leaked answer. TTT changed the reasoning style (more confident, more willing to shortcut), not the capability.

A.4.4 D.4 The opposite stylistic drift (idx8 student eval, PRE vs POST)

idx8 drifts the other way. **PRE** (boxed **40201**) treats the rolls as independent and computes $p = 2^{13} \cdot 3/5^6$, giving $m + n = 24576 + 15625 = 40201$. **POST** (boxed **17**) is *more* rigorous — it does casework on $R_2 = R_4$ versus $R_2 \neq R_4$, counts $N_B = 11700$ and $N_{A \cap B} = 3600$, and simplifies $p = 4/13$ to $m + n = 17$. POST is longer and more careful, yet still wrong (correct 29). Across the two problems, then, TTT moves the reasoning style in opposite directions (idx9 lazier, idx8 more rigorous) while leaving correctness unchanged.

A.4.5 D.5 Genuine discovery on code (idx12, run `teacherfirst-...-idx12`)

The contrast with the math pilot is the point of the thesis. Here the output is a *program*, and teacher-first installs a working one. Problem `abc389_b`: given X, find N with $N! = X$.

PRE (greedy score 0.075, pass@16 = 0.000). The student inverts the task, computing a factorial instead of finding N, which also overflows the integer-string limit on large inputs:

```
import math
X = int(input())
print(math.factorial(X))
```

Execution feedback (the privileged context — this is what "execution-guided" means):

```
Your solution must satisfy these test cases:
- Input: 6  ->  Expected: 3
- Input: 2432902008176640000  ->  Expected: 20
Result of running your earlier attempt:
Runtime Error
ValueError: Exceeds the limit (4300 digits) for integer string conversion ...
Last Executed Input: 5040
```

POST (greedy score 1.000, pass@16 = 1.000). The student now iterates to find N, a correct algorithm that generalizes to any input:

```
import sys
def find_n_factorial(X):
```

```
    n = 1
    current = 1
    while current < X:
        n += 1
        current *= n
    return n
def main():
    X = int(sys.stdin.readline())
    print(find_n_factorial(X))
if __name__ == "__main__":
    main()
```

PRE pass 0.000 → POST pass 1.000 (discovery curve 0.85 → 1.0 → … → 1.0; greedy 0.075 → 1.0). Unlike the math pilot, the distilled output is a method: a program that solves the problem for every input, not a value to copy. This is the value-versus-procedure boundary (Chapter 5) made concrete.

A.5 Appendix E — Per-seed results

A.5.1 E.1 idx39 (abc393_d, hard, base pass ≈ 0.1), eval-16, diff-flib judge

seed	PRE	TF POST	SF POST
0	0.000	0.438	0.188
1	0.000	0.062	0.000
2	0.062	0.125	0.125
3	0.188	0.062	0.062
mean		0.172	0.094

A.5.2 E.2 idx12 (abc389_b, model-hard, model pass ≈ 0.12), eval-16

seed	PRE	TF POST	SF POST
0	0.000	1.000	0.938
1	0.062	1.000	0.500
2	0.062	1.000	1.000
3	0.125	1.000	0.938
mean		1.000	0.844

A.5.3 E.3 Hard-frontier means and escape-zero instances

problem	id	judge	SF mean	TF mean	win/tie/loss	escape-zero seeds
idx39	abc393_d	difflib	0.094	0.172	2/2/0	s1 (SF 0→0, TF 0.062)
idx64	abc397_e	LLM-groq	0.047	0.422	4/0/0	s1 (SF 0.062→0, TF 0.375), s2 (SF 0→0, TF 0.062)
idx77	abc399_f	difflib	0.203	0.344	3/1/0	—

Per-seed POST pass@16 for idx64 and idx77, recovered from the W&B export and reconciled to the published means (canonical runs: idx64 = LLM judge, idx77 = difflib, both good_only / T2; dedup to the latest run per seed; `data/reconcile.py`):

seed	idx64 TF	idx64 SF	idx77 TF	idx77 SF
0	0.5625	0.000	0.250	0.0625
1	0.375	0.0625	0.3125	0.250
2	0.0625	0.000	0.6875	0.375
3	0.625	0.125	0.125	0.125
mean	0.406	0.047	0.344	0.203

The idx64 TF mean from these canonical runs is 0.406 versus the 0.422 reported in §4.2.3; the difference is which re-run is selected. Note also that the canonical idx64 seed-1 SF run here is 0.0625, whereas the escape-zero instance in §4.2.3 (SF → 0) is a different run of the same seed: the stuck-at-zero behavior is **run-dependent** (§6.1.1).

A.5.4 E.4 RQ1 template (SF arm, 2 seeds), POST pass@16

Template	idx12 (easy)	idx39 (hard)
T1_minimal	0.66	0.03
T2_standard	0.97	0.06
T5_reasoning	0.97	0.13

T5 on idx39 is stable across its two seeds (.125 / .125).

A.5.5 E.5 Judge × few-shot ablation, mean POST pass (4 seeds)

arm	idx39	idx12
SF baseline	0.094	0.844
TF · good_only · difflib	0.172	1.000
TF · good_only · LLM	0.266	0.984

arm	idx39	idx12
TF · good_bad · LLM	0.250	1.000

A.6 Appendix F — Frontier scans

A.6.1 F.1 Math (AIME 2026, Gemma-4-E4B thinking ON, n_samples = 2)

Band	Problem indices
Frontier ($0 <$ pass < 1)	8, 11, 21, 25
Too-hard (pass = 0)	2, 3, 9, 10, 12, 13, 14, 16, 17, 22, 24, 26, 27, 28, 29
Ceiling (pass = 1)	0, 1, 4, 5, 6, 7, 15, 18, 19, 20, 23

The two pilot problems (idx9, idx8) are drawn from the too-hard band; idx8 was labeled frontier by the noisy 2-sample scan but is actually pass = 0 at 4 samples.

A.6.2 F.2 Code (LiveCodeBench v6, Qwen3-4B)

Problems for the matched comparisons were selected by *model* pass-rate $\in (0, 1)$ over a scan of idx0–90. The problems used are idx12 (abc389_b), idx39 (abc393_d), idx64 (abc397_e), and idx77 (abc399_f).

> **To be completed:** the full idx0–90 model-pass-rate scan table is in the logs and should be reproduced here for completeness.